

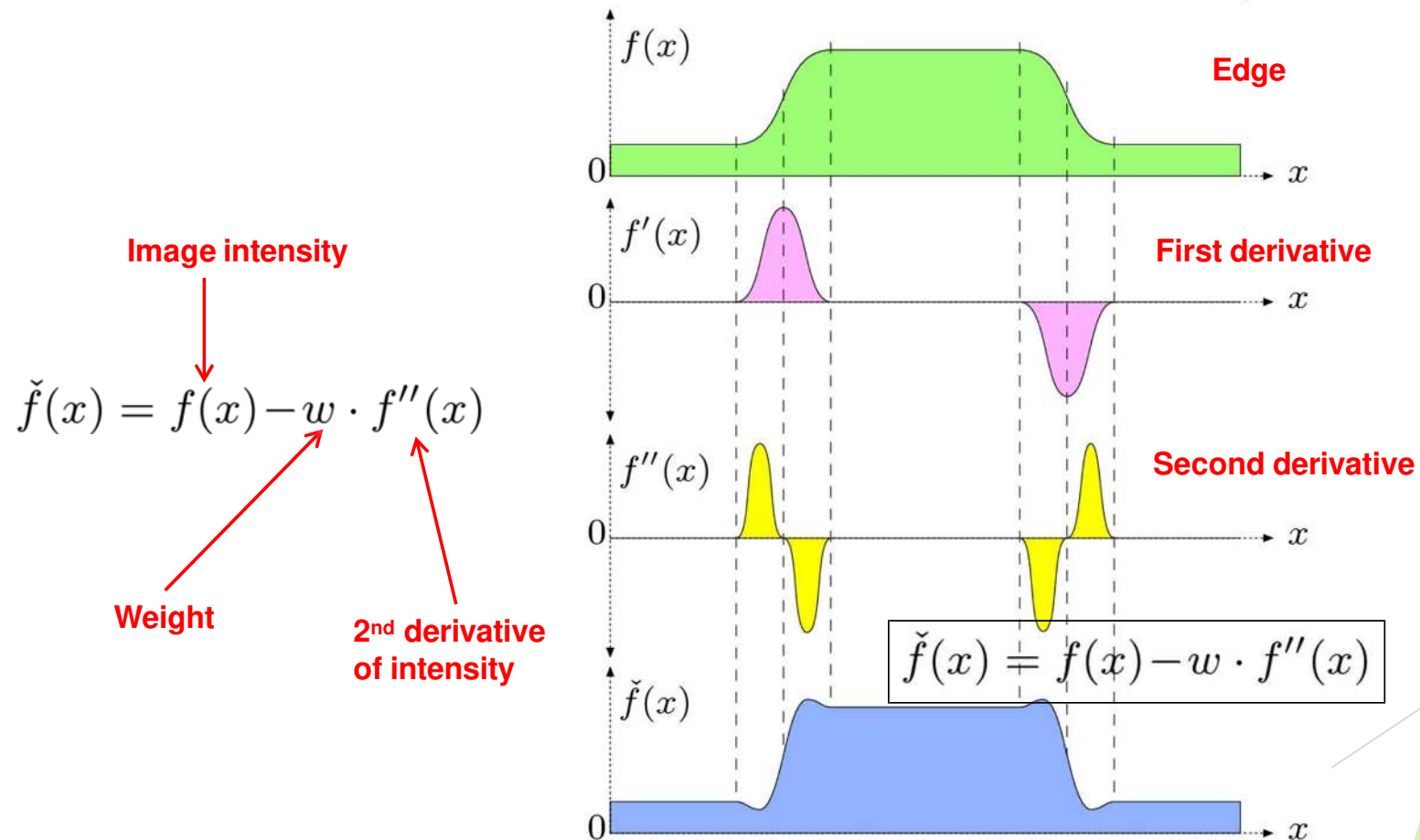
Sayısal Görüntü İşleme

Prof Dr. Gökhan BİLGİN
Yıldız Teknik Üniversitesi
Bilgisayar Mühendisliği Bölümü

Image Sharpening

- Blurring may occur during image scanning or scaling
- Sharpening reduces effects of blurring
- How? Amplify high frequency components
- High frequencies occur at edges
- We need to sharpen edges
- Two main approaches:
 - Using Laplace filter
 - Unsharp masking

Edge Sharpening using Laplace Filter



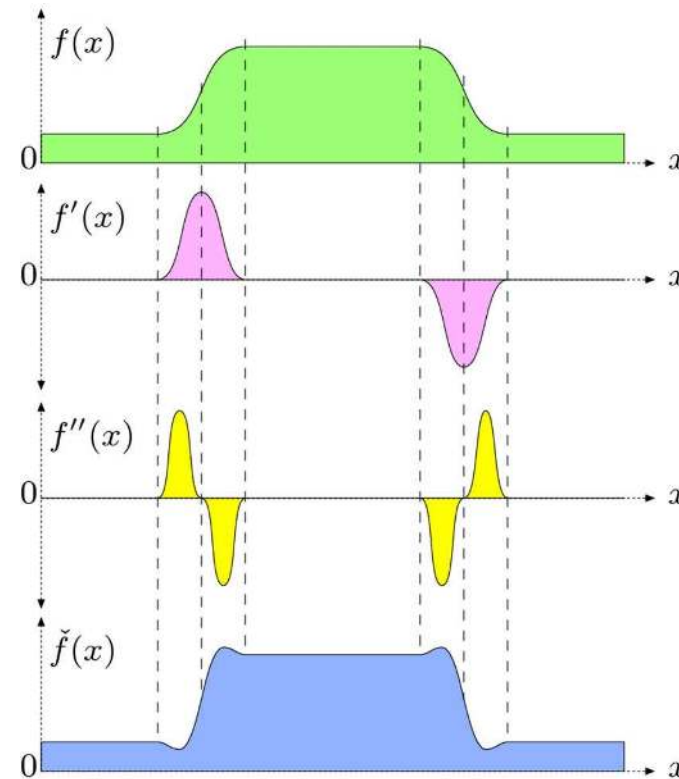
Laplace Operator

- **2D Laplace operator:** combines 2nd derivatives in horizontal and vertical directions
- Laplace operator defined as:

$$(\nabla^2 f)(x, y) = \frac{\partial^2 f}{\partial^2 x}(x, y) + \frac{\partial^2 f}{\partial^2 y}(x, y)$$

2nd derivative
of intensity in
x direction

2nd derivative
of intensity in
y direction

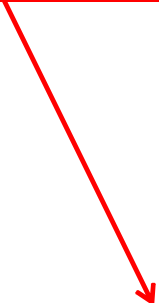


Laplacian Operator

- Laplacian: $(\nabla^2 f)(x, y) = \frac{\partial^2 f}{\partial^2 x}(x, y) + \frac{\partial^2 f}{\partial^2 y}(x, y)$
- Digital approximation of laplacian is:

$$\nabla^2 f(x, y) = [f(x + 1, y) - f(x, y)] - [f(x, y) - f(x - 1, y)] + \\ [f(x, y + 1) - f(x, y)] - [f(x, y) - f(x, y - 1)]$$

$$= [f(x + 1, y) + f(x - 1, y) + f(x, y + 1) - f(x, y - 1)] - 4f(x, y)$$



0	1	0
1	-4	1
0	1	0

Laplacian Operator

- Laplacian: $(\nabla^2 f)(x, y) = \frac{\partial^2 f}{\partial^2 x}(x, y) + \frac{\partial^2 f}{\partial^2 y}(x, y)$

1d filters that estimate 2nd derivatives along x and y directions

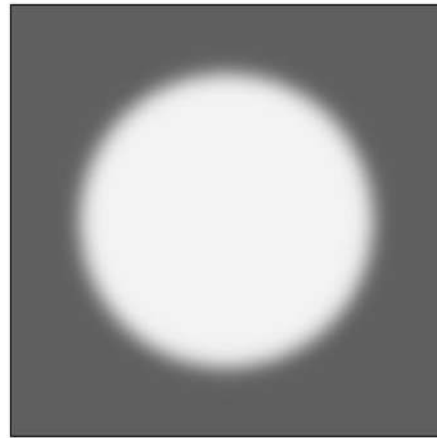
$$\frac{\partial^2 f}{\partial^2 x} \equiv H_x^L = [1 \quad -2 \quad 1] \quad \text{and} \quad \frac{\partial^2 f}{\partial^2 y} \equiv H_y^L = \begin{bmatrix} 1 \\ -2 \\ 1 \end{bmatrix}$$

$$H^L = H_x^L + H_y^L = \begin{bmatrix} 0 & 1 & 0 \\ 1 & -4 & 1 \\ 0 & 1 & 0 \end{bmatrix}$$

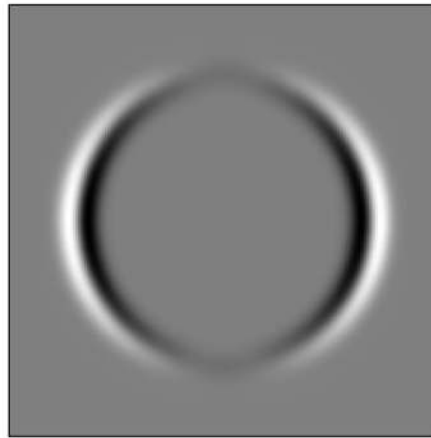
2-dimensional Laplace filter

Results for Laplacian Operator

Synthetic test
Image /



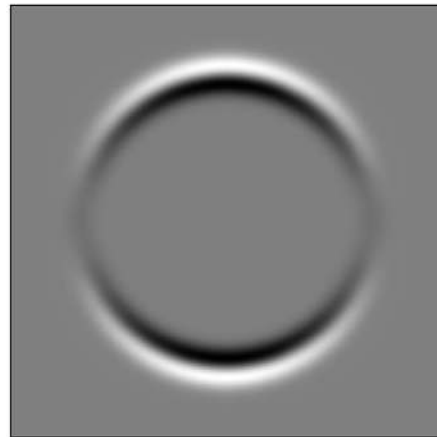
(a)



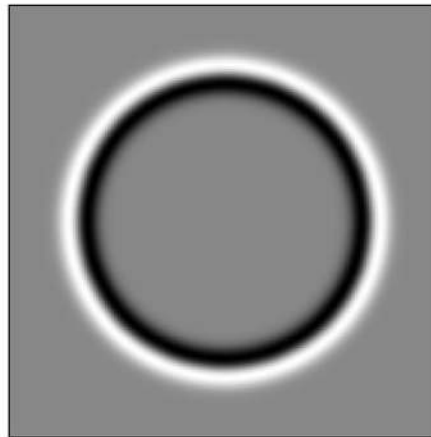
(b)

Second partial
derivative
in horizontal
direction

Second partial
derivative in
Vertical direction



(c)



(d)

Laplace filter

Edge Sharpening

- To perform actual sharpening:
 - First apply Laplace filter to Image I
 - Subtract fraction of result from original image

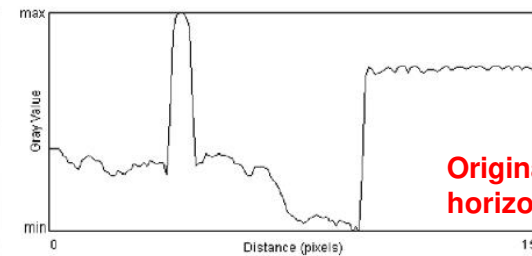
$$\check{I} \leftarrow I - w \cdot (H^L * I)$$

$$H^L = H_x^L + H_y^L = \begin{bmatrix} 0 & 1 & 0 \\ 1 & -4 & 1 \\ 0 & 1 & 0 \end{bmatrix}$$

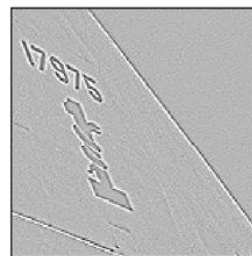
2-dimensional Laplace filter



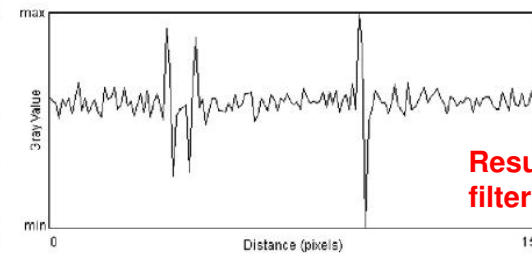
(a)



Original image with horizontal profile



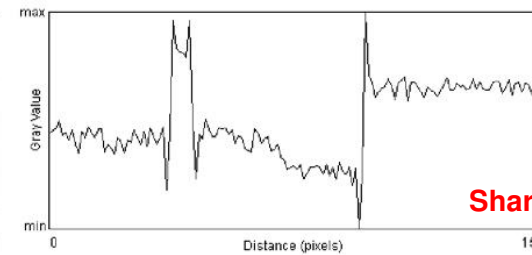
(c)



Result of Laplace filter H^L



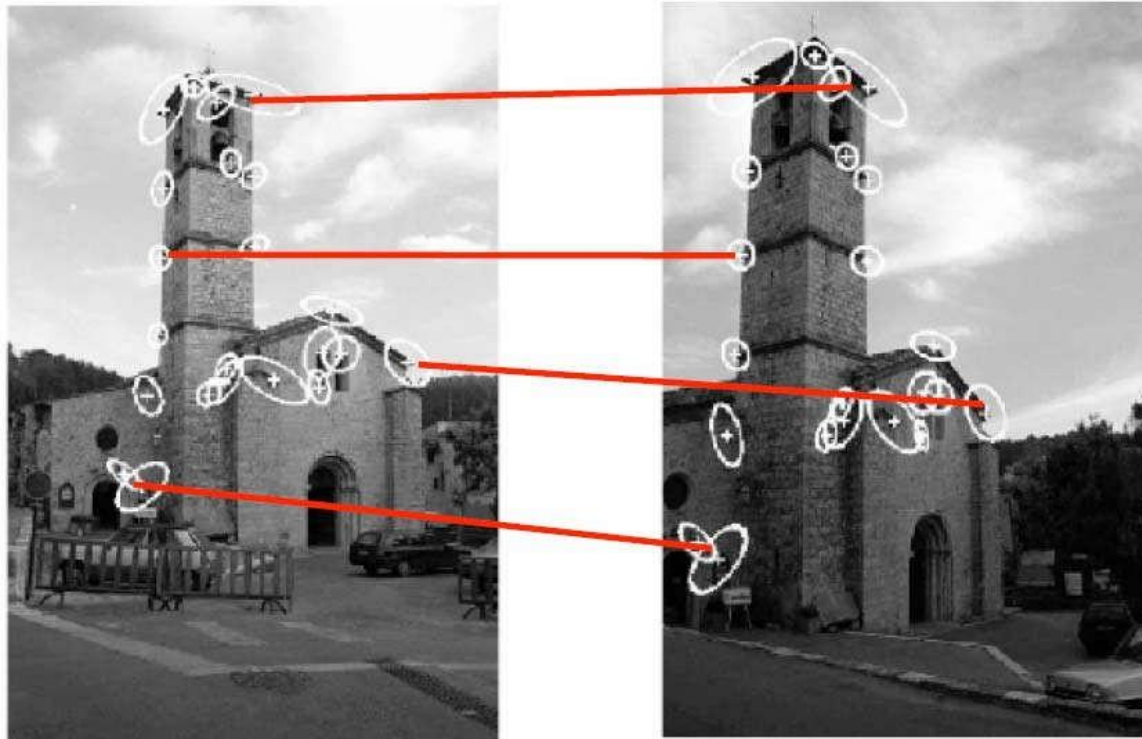
(e)



Sharpened Image

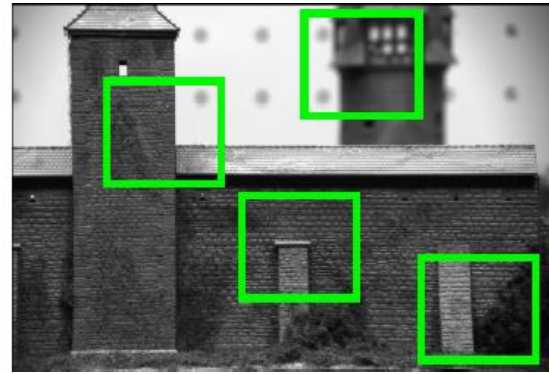
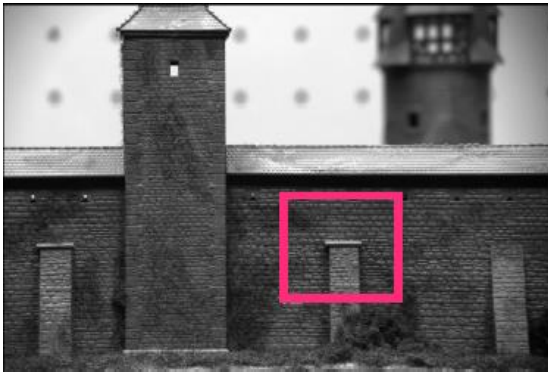
Why Corner Detection?

- Corners are robust, used in computer vision for matching same point in multiple images (e.g. stereo left + right image)



Motivation: Patch Matching

- Consider small squares called **patches** of same size in both images

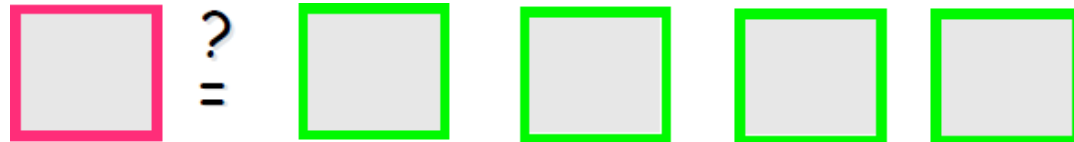


- The task?** Find most similar patch in second image



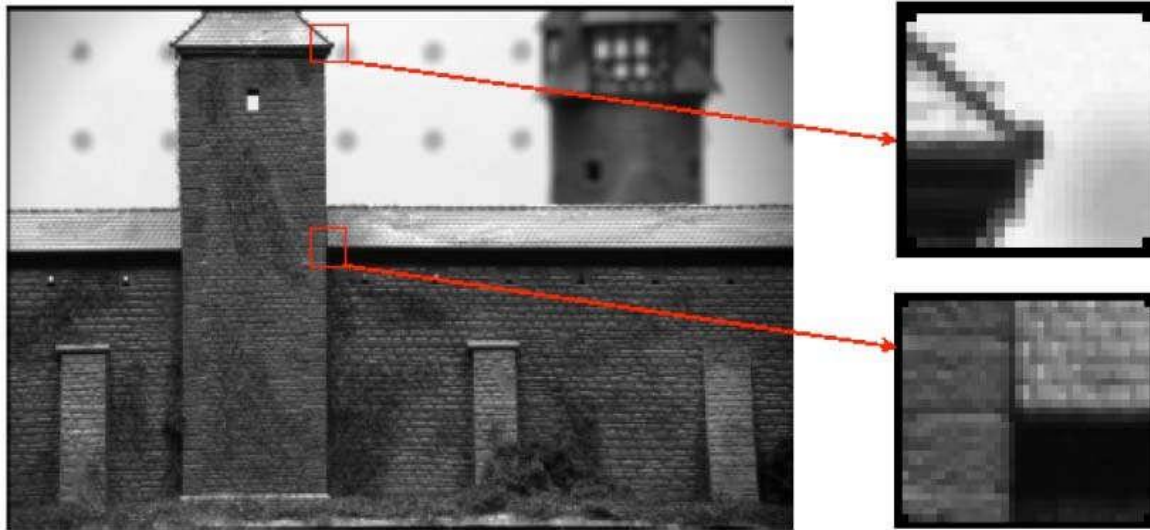
Some Patches Better Than Others

- More robust to use **distinctive patches**
- Example: Don't use patches similar to many patches in image 2 (ambiguous)



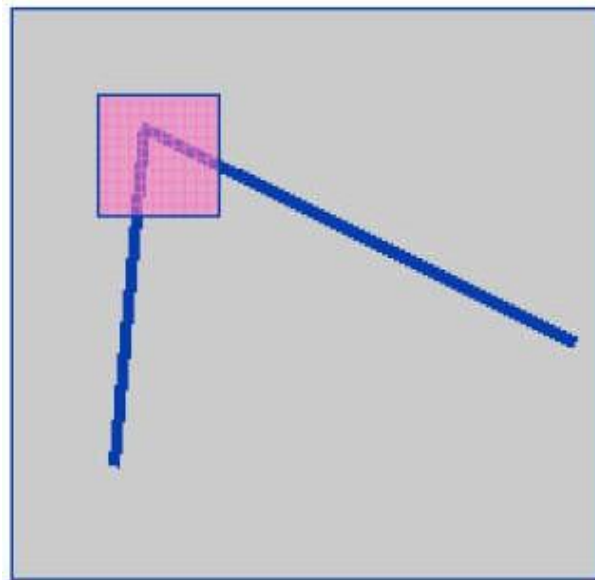
Corners are Robust “Features”

- Corners are unique, match patches with corners
- What are corners? Junctions of contours
- Corners appear as large changes in intensity in different viewpoints (stable/unique)

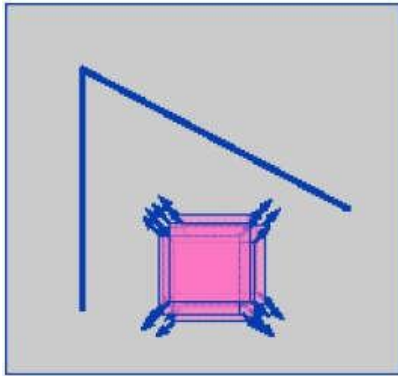


Corner Points: Basic Idea

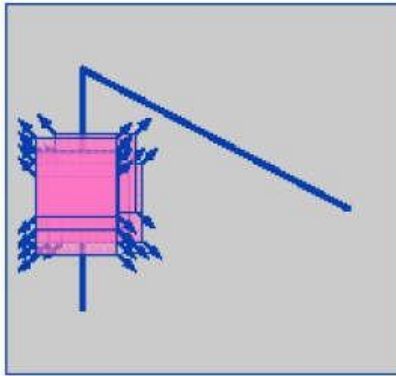
- **Patch at corner:** shifting window in any direction yields large change in intensity
- **Is patch at corner?** shift window in multiple directions, if large intensity changes, patch is a corner



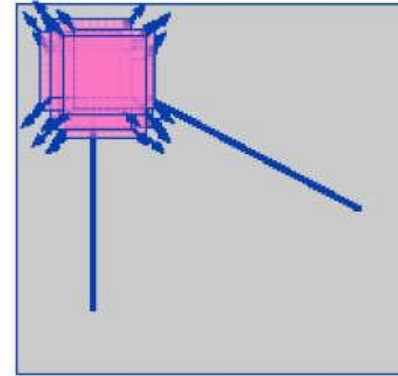
Harris Corner Detector: Basic Idea



Flat region: no intensity change in all directions



Edge: no change along edge directions

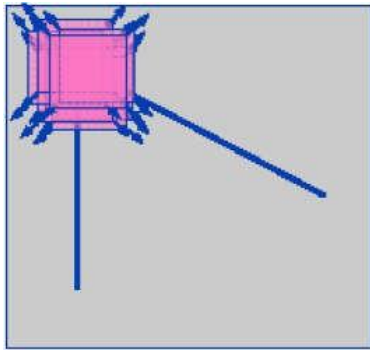


Corner: significant intensity change in many directions

- Harris corner detector gives mathematical approach for determining which case holds

Harris Detector: The Mathematics

- Shift patch by $[u, v]$ and compute change in intensity
- Change of intensity for shift $[u, v]$



$$E(u, v) = \sum_{x, y} w(x, y) [I(x+u, y+v) - I(x, y)]^2$$

Window
function

Shifted
intensity

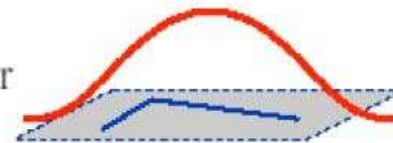
Intensity

Window function $w(x, y) =$



1 in window, 0 outside

or



Gaussian

Harris Detector: The Intuition

- Change of intensity for shift $[u, v]$

$$E(u, v) = \sum_{x, y} w(x, y) [I(x+u, y+v) - I(x, y)]^2$$

The diagram shows the Harris detector equation $E(u, v) = \sum_{x, y} w(x, y) [I(x+u, y+v) - I(x, y)]^2$ on a green background. A red oval highlights the term $[I(x+u, y+v) - I(x, y)]^2$. Three blue arrows point from labels below to parts of the equation: 'Window function' points to $w(x, y)$, 'Shifted intensity' points to $I(x+u, y+v)$, and 'Intensity' points to $I(x, y)$. A red arrow points from the highlighted term to a text box below.

For nearly flat regions: this quantity almost 0
For distinctive (corner) : this quantity almost large
Hence... Corners? Patches where $E(u, v)$ is **LARGE**

Taylor Series Expansion

$$f(x+u, y+v) = f(x, y) + uf_x(x, y) + vf_y(x, y) +$$

First partial derivatives

$$\frac{1}{2!} [u^2 f_{xx}(x, y) + uv f_{xy}(x, y) + v^2 f_{yy}(x, y)] +$$

Second partial derivatives

$$\frac{1}{3!} [u^3 f_{xxx}(x, y) + u^2 v f_{xxy}(x, y) + uv^2 f_{xyy}(x, y) + v^3 f_{yyy}(x, y)]$$

Third partial derivatives

+ ... (Higher order terms)

$$E(u, v) = \sum_{x, y} w(x, y) [I(x+u, y+v) - I(x, y)]^2$$

Window
function

Shifted
intensity

Intensity

$$\approx \sum [I(x, y) + uI_x + vI_y - I(x, y)]^2$$

**First order
approximation**

Harris Corner Derivation

$$\sum [I(x+u, y+v) - I(x, y)]^2$$

$$\approx \sum [I(x, y) + uI_x + vI_y - I(x, y)]^2 \quad \text{First order approx}$$

$$= \sum u^2 I_x^2 + 2uv I_x I_y + v^2 I_y^2$$

$$= \sum [u \ v] \begin{bmatrix} I_x^2 & I_x I_y \\ I_x I_y & I_y^2 \end{bmatrix} \begin{bmatrix} u \\ v \end{bmatrix} \quad \text{Rewrite as matrix equation}$$

$$= [u \ v] \left(\sum \begin{bmatrix} I_x^2 & I_x I_y \\ I_x I_y & I_y^2 \end{bmatrix} \right) \begin{bmatrix} u \\ v \end{bmatrix}$$

Harris Corner Detector

- For small shifts $[u, v]$, we have following approximation

$$E(u, v) \cong [u, v] M \begin{bmatrix} u \\ v \end{bmatrix}$$

where M is the 2x2 matrix

$$M = \sum_{x,y} w(x, y) \begin{bmatrix} I_x^2 & I_x I_y \\ I_x I_y & I_y^2 \end{bmatrix}$$

Harris Corner Detector

$$M = \sum_{x,y} w(x,y) \begin{bmatrix} I_x^2 & I_x I_y \\ I_x I_y & I_y^2 \end{bmatrix}$$

- If we compute derivatives $A(u,v)$, $B(u,v)$ and $C(u,v)$

$$A(u,v) = I_x^2(u,v)$$

$$B(u,v) = I_y^2(u,v)$$

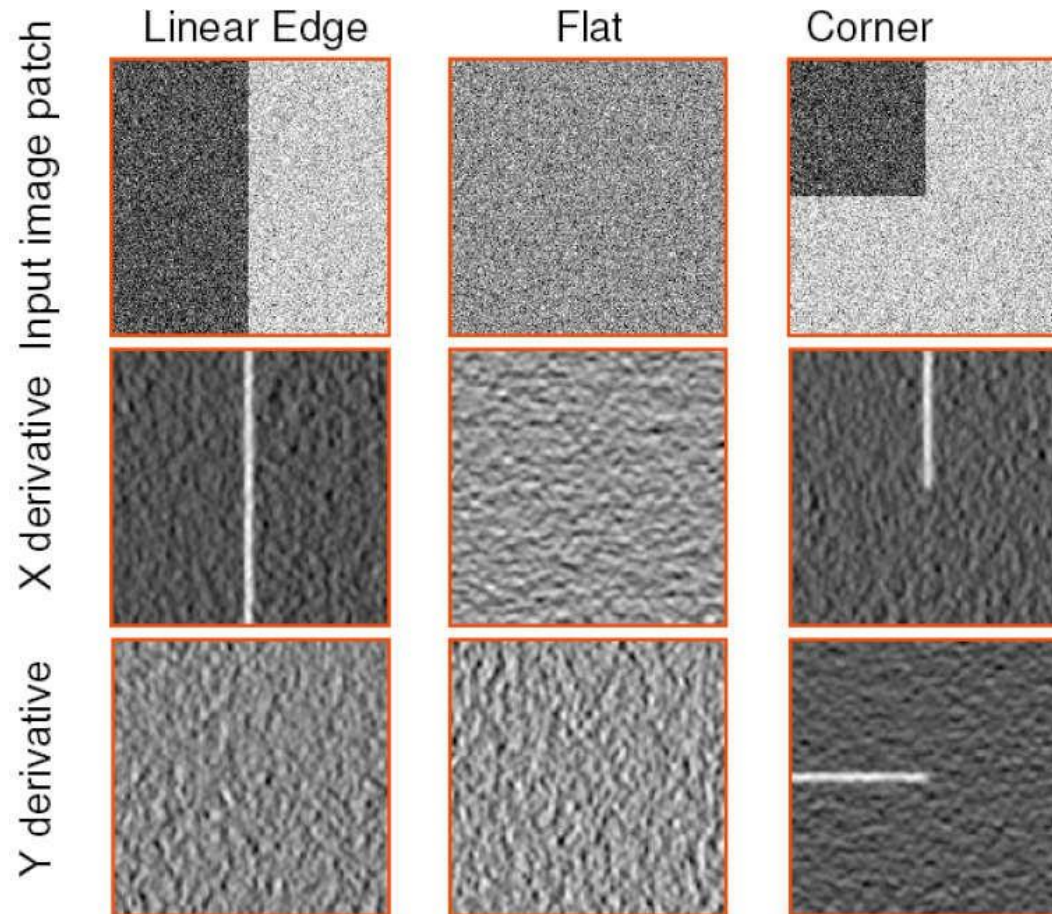
$$C(u,v) = I_x(u,v) \cdot I_y(u,v)$$

- Can express matrix M as

$$M = \begin{pmatrix} I_x^2 & I_x I_y \\ I_x I_y & I_y^2 \end{pmatrix} = \begin{pmatrix} A & C \\ C & B \end{pmatrix}$$

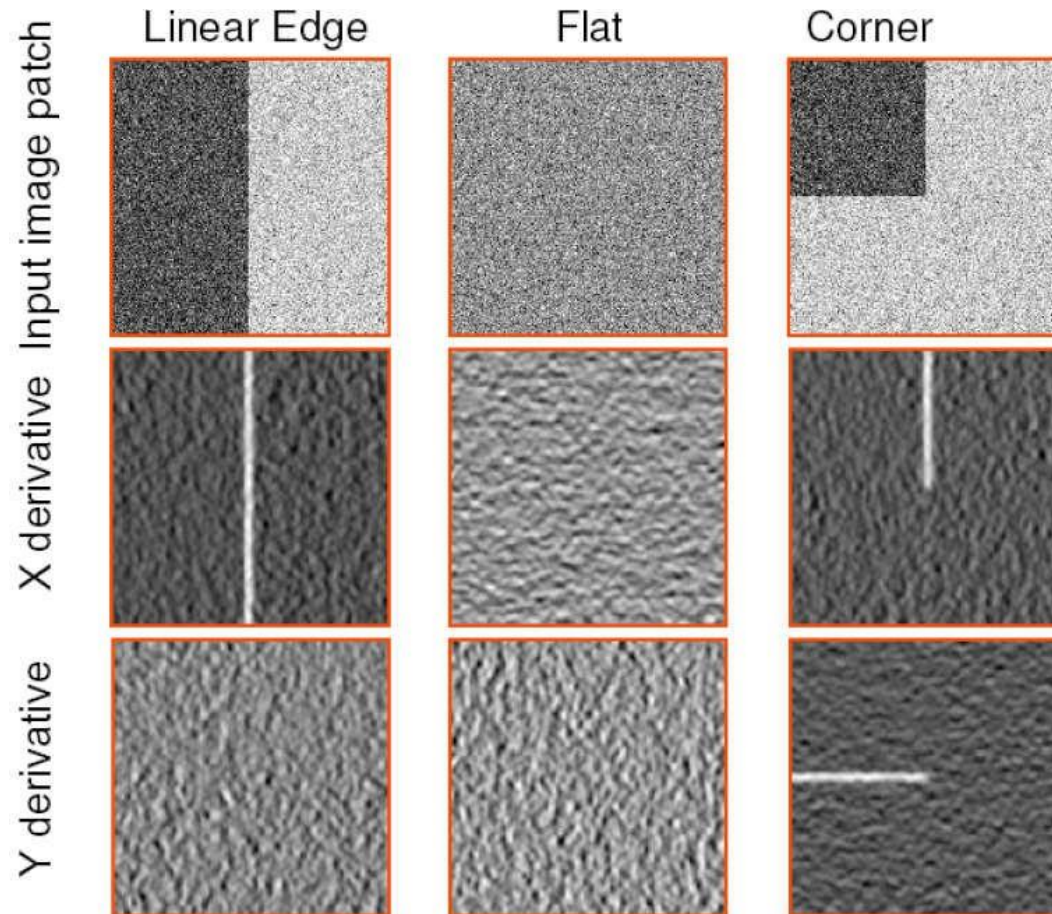
Harris Corner Detection Intuition

Example: Cases and 2D Derivatives



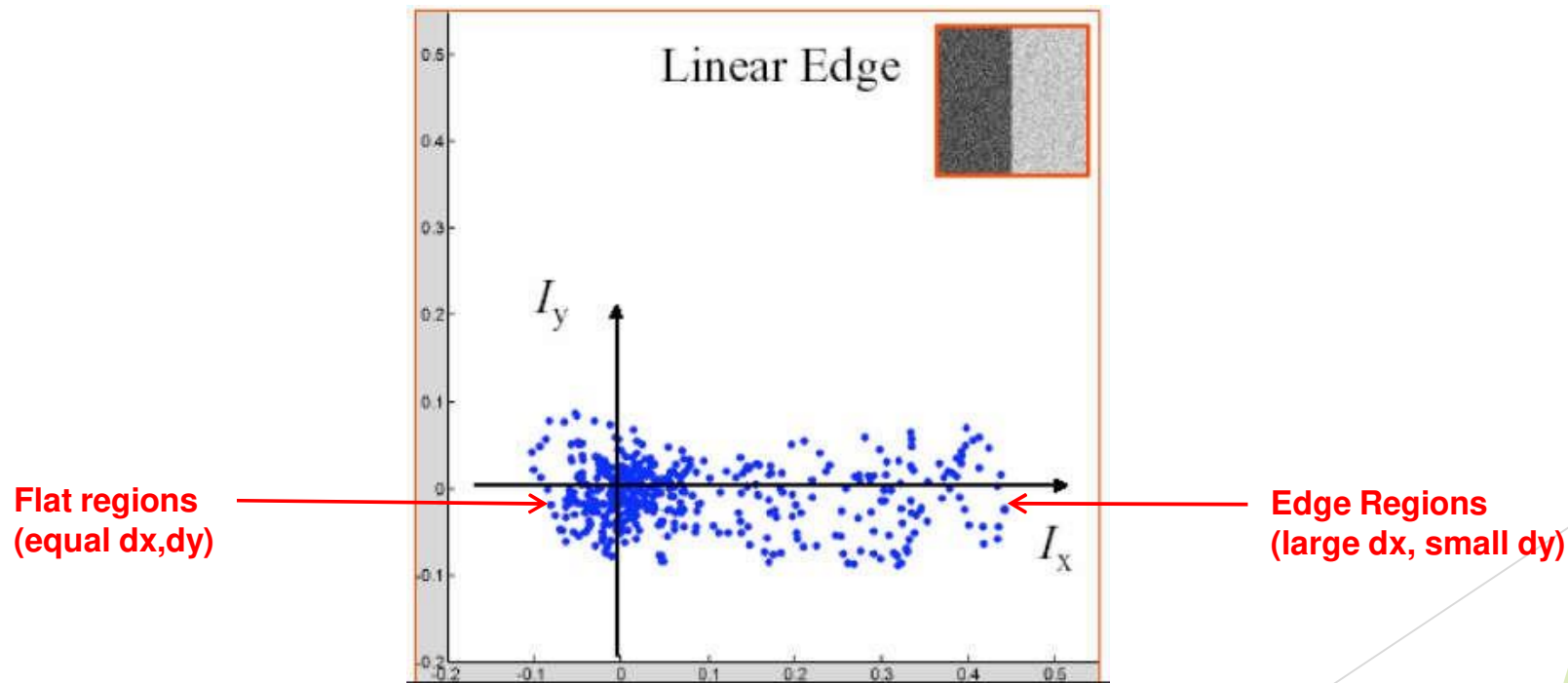
Harris Corner Detection Intuition

Example: Cases and 2D Derivatives

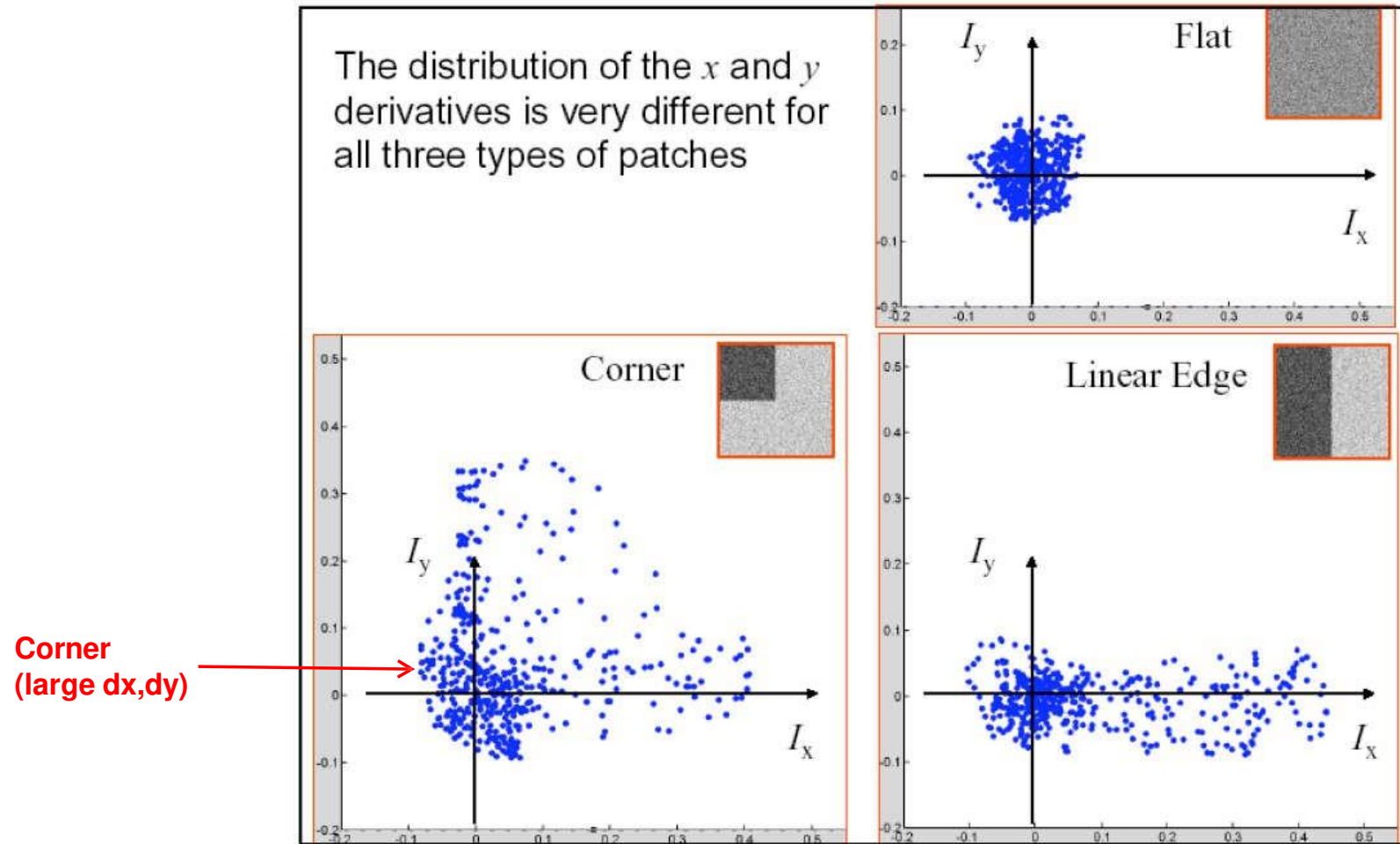


Some Intuition: Plotting Derivatives as 2D Points

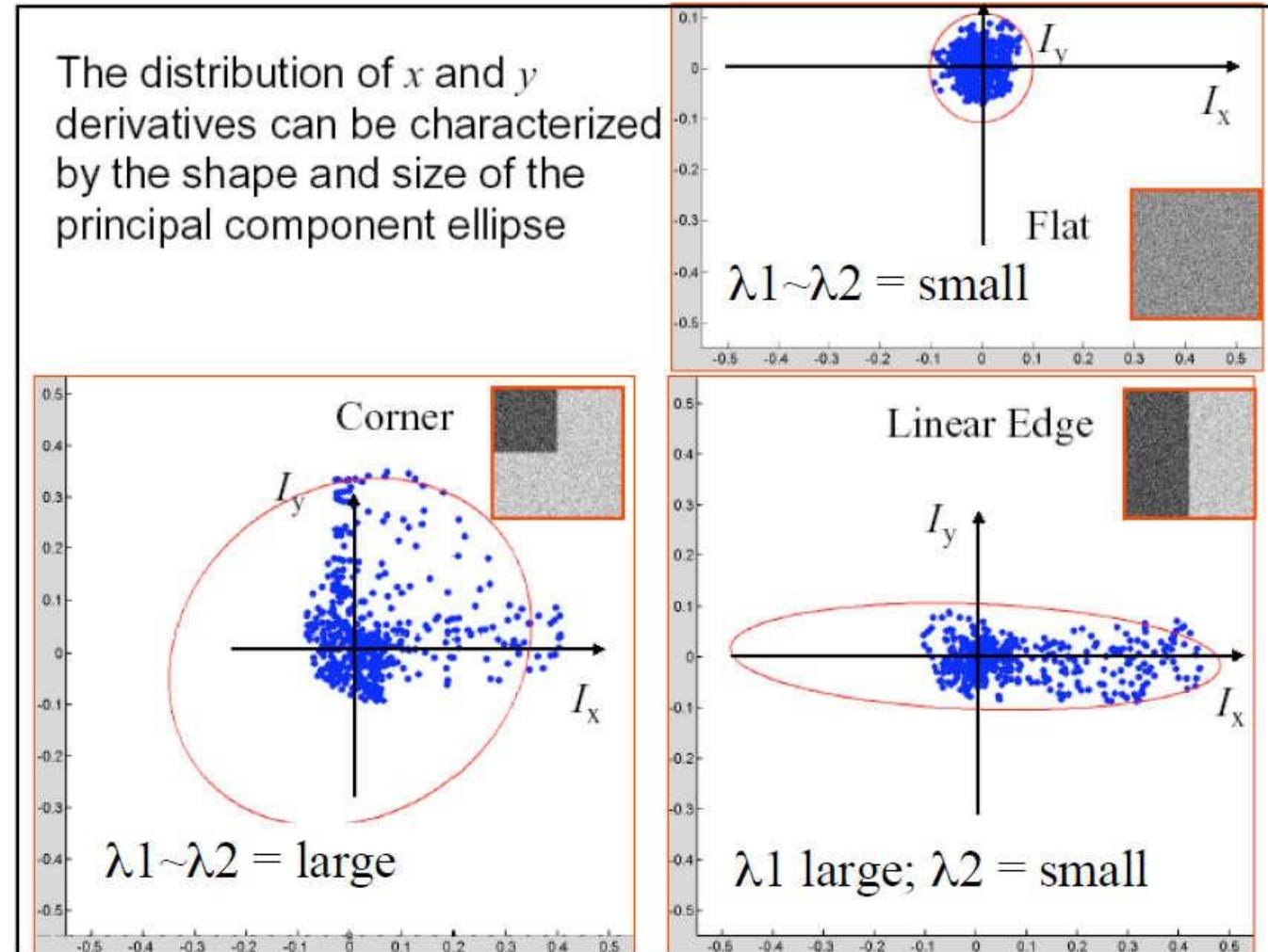
- Treat gradient vectors as set of (dx, dy) points with center at $(0,0)$
- Fit an ellipse to that set of points via scatter matrix
- Analyze ellipse parameters for varying cases...



Plotting Derivatives as 2D Points



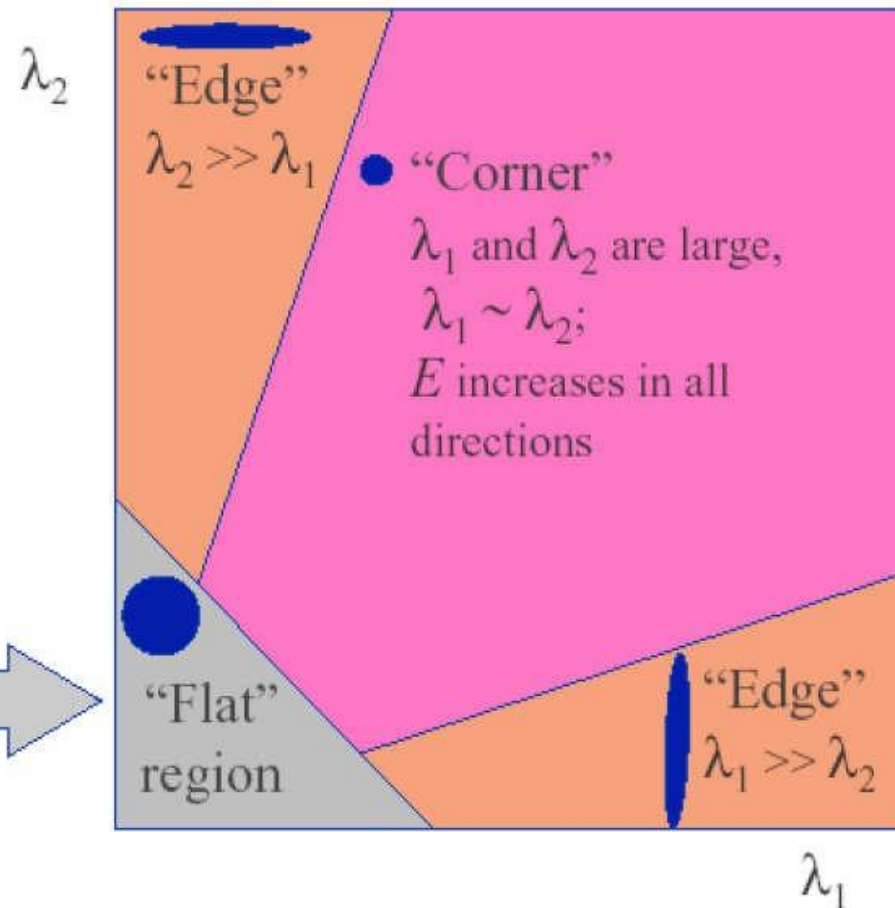
Fitting Ellipse to Each Set of Points



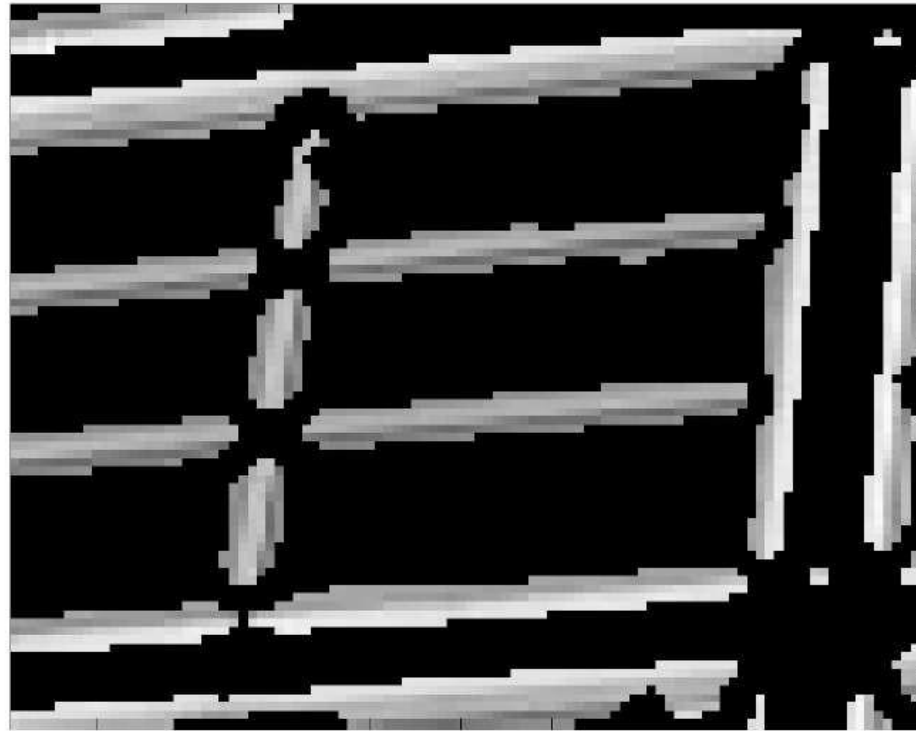
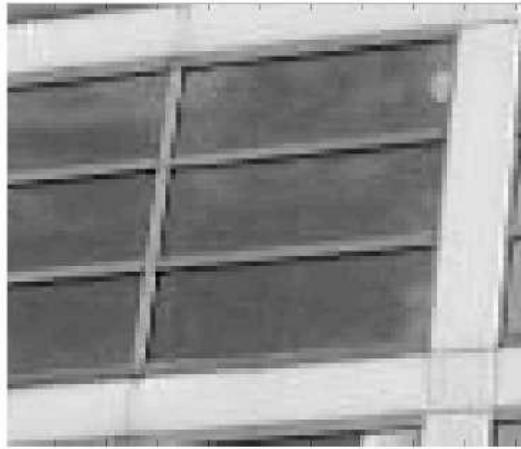
Classification of Eigenvalues

Classification of
image points using
eigenvalues of M :

λ_1 and λ_2 are small;
 E is almost constant
in all directions

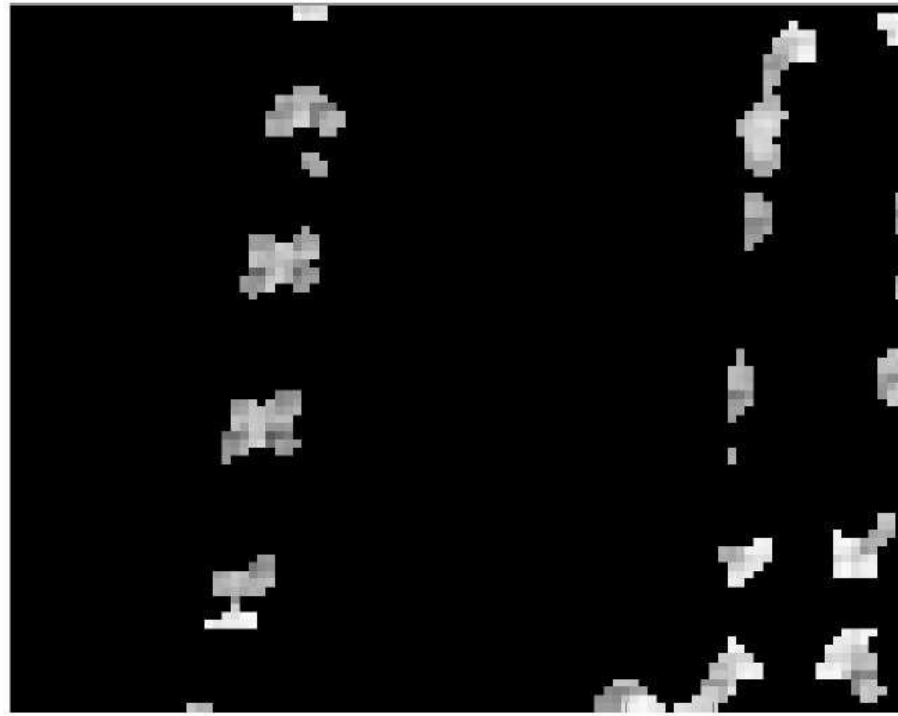
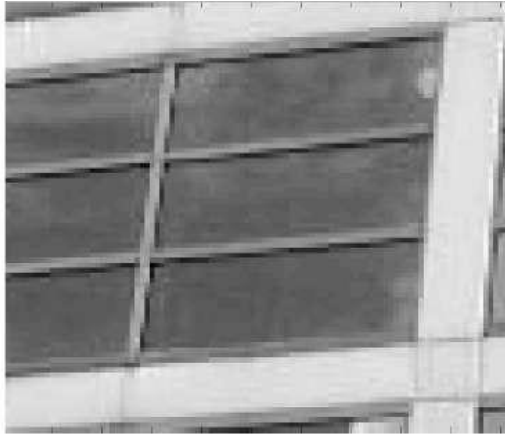


Harris Corner Response Example



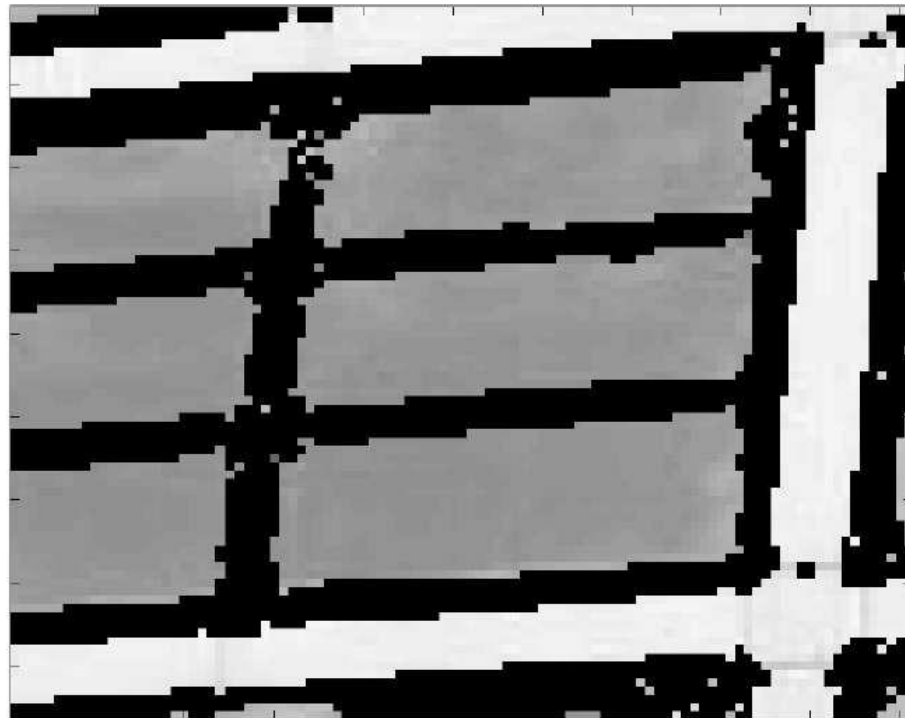
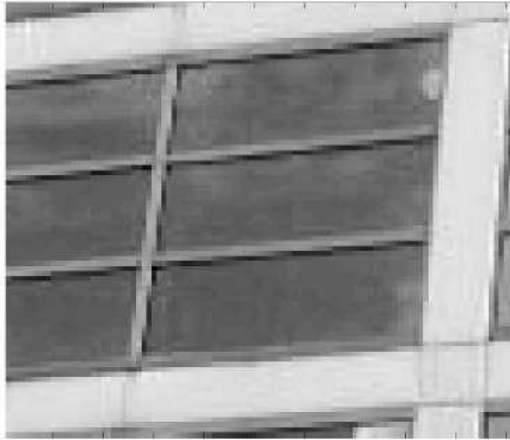
Threshold: $R < -10000$
(edges)

Harris Corner Response Example



Threshold: > 10000
(corners)

Harris Corner Response Example



Threshold: $-10000 < R < 10000$
(neither edges nor corners)

Harris Corner Detector

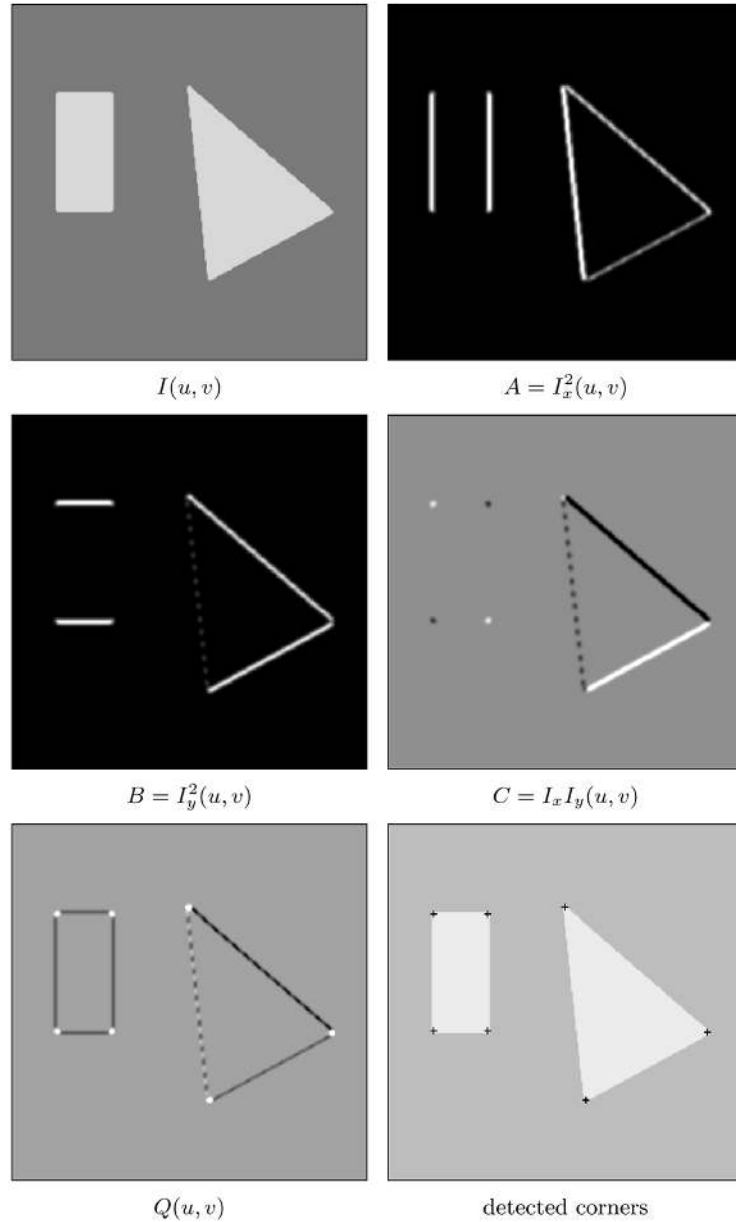
- An image location (u,v) is candidate for corner point when

$$Q(u, v) > t_H$$

- t_H is threshold selected based on image content, typically lies in range 10,000 – 1,000,000
- Detected corners inserted into set and sorted in descending order (i.e. $q_i \geq q_{i+1}$) based on their corner strength

$$Corners = [c_1, c_2, \dots, c_N]$$

- Many false corners occur in neighborhood of real corner
- Traverse sorted list, delete false corners towards end of list

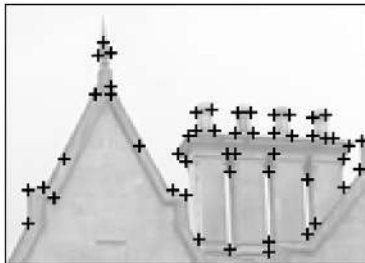


Harris Corner Detection Example (Synthetic Image)

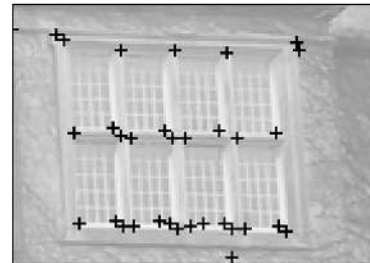


(a)

Image with final
corner points marked

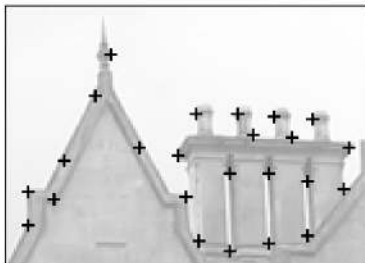


(b)

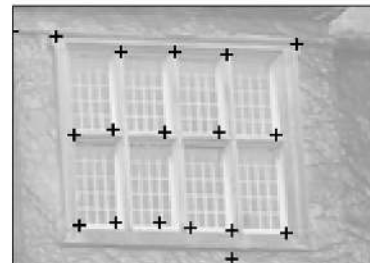


(c)

Before thresholding values



(d)



(e)

After thresholding

Harris Corner
Detection
Example (Real
Image)

```

1: HARRISCORNERS( $I$ )
   Returns a list of the strongest corners found in the image  $I$ .

2: STEP 1—COMPUTE THE CORNER RESPONSE FUNCTION:
3:   Prefilter (smooth) the original image:  $I' \leftarrow I * H_p$ 
4:   Compute the horizontal and vertical image derivatives:
        $I_x \leftarrow I' * H_{dx}$ 
        $I_y \leftarrow I' * H_{dy}$ 
5:   Compute the local structure matrix  $M(u, v) = \begin{pmatrix} A & C \\ C & B \end{pmatrix}$ :
        $A(u, v) \leftarrow I_x^2(u, v)$ 
        $B(u, v) \leftarrow I_y^2(u, v)$ 
        $C(u, v) \leftarrow I_x(u, v) \cdot I_y(u, v)$ 
6:   Blur each component of the structure matrix:  $\bar{M} = \begin{pmatrix} \bar{A} & \bar{C} \\ \bar{C} & \bar{B} \end{pmatrix}$ :
        $\bar{A} \leftarrow A * H_b$ 
        $\bar{B} \leftarrow B * H_b$ 
        $\bar{C} \leftarrow C * H_b$ 
7:   Compute the corner response function:
        $Q \leftarrow (\bar{A} \cdot \bar{B} - \bar{C}^2) - \alpha \cdot (\bar{A} + \bar{B})^2$ 

8: STEP 2—COLLECT CORNER POINTS:
9:   Create an empty list:
        $Corners \leftarrow []$ 
10:  for all image coordinates  $(u, v)$  do
11:    if  $Q(u, v) > t_H$  and ISLOCALMAX( $Q, u, v$ ) then
12:      Create a new corner:
          $c_i \leftarrow \langle u_i, v_i, q_i \rangle = \langle u, v, Q(u, v) \rangle$ 
13:      Add  $c_i$  to  $Corners$ 
14:  Sort  $Corners$  by  $q_i$  in descending order (strongest corners first)
15:   $GoodCorners \leftarrow \text{CLEANUPNEIGHBORS}(Corners)$ 
16:  return  $GoodCorners$ .

```

```

17: ISLOCALMAX( $Q, u, v$ )    ▷ determine if  $Q(u, v)$  is a local maximum
18:   Let  $q_c \leftarrow Q(u, v)$  (center pixel)
19:   Let  $\mathcal{N} \leftarrow \text{Neighbors}(Q, u, v)$     ▷ values of all neighboring pixels
20:   if  $q_c \geq q_i$  for all  $q_i \in \mathcal{N}$  then
21:     return true
22:   else
23:     return false.

24: CLEANUPNEIGHBORS( $Corners$ )  ▷  $Corners$  is sorted by descending  $q$ 
25:   Create an empty list:
        $GoodCorners \leftarrow []$ 
26:   while  $Corners$  is not empty do
27:      $c_i \leftarrow \text{REMOVEFIRST}(Corners)$ 
28:     Add  $c_i$  to  $GoodCorners$ 
29:     for all  $c_j$  in  $Corners$  do
30:       if  $\text{Dist}(c_i, c_j) < d_{\min}$  then
31:         Delete  $c_j$  from  $Corners$ 
32:   return  $GoodCorners$ .

```

Harris Corner Detection Algorithm

Harris Corner Detection Algorithm & Parameters

```

1: HARRISCORNERS(I)
   Returns a list of the strongest corners found in the image I.

2: STEP 1—COMPUTE THE CORNER RESPONSE FUNCTION:
3:   Prefilter (smooth) the original image:  $I' \leftarrow I * H_p$ 
4:   Compute the horizontal and vertical image derivatives:
        $I_x \leftarrow I' * H_{dx}$ 
        $I_y \leftarrow I' * H_{dy}$ 
5:   Compute the local structure matrix  $M(u, v) = \begin{pmatrix} A & C \\ C & B \end{pmatrix}$ :
        $A(u, v) \leftarrow I_x^2(u, v)$ 
        $B(u, v) \leftarrow I_y^2(u, v)$ 
        $C(u, v) \leftarrow I_x(u, v) \cdot I_y(u, v)$ 
6:   Blur each component of the structure matrix:  $\tilde{M} = \begin{pmatrix} \tilde{A} & \tilde{C} \\ \tilde{C} & \tilde{B} \end{pmatrix}$ :
        $\tilde{A} \leftarrow A * H_b$ 
        $\tilde{B} \leftarrow B * H_b$ 
        $\tilde{C} \leftarrow C * H_b$ 
7:   Compute the corner response function:
        $Q \leftarrow (\tilde{A} \cdot \tilde{B} - \tilde{C}^2) - \alpha \cdot (\tilde{A} + \tilde{B})^2$ 

8: STEP 2—COLLECT CORNER POINTS:
9:   Create an empty list:
       Corners  $\leftarrow []$ 
10:  for all image coordinates (u, v) do
11:    if  $Q(u, v) > t_H$  and ISLOCALMAX(Q, u, v) then
12:      Create a new corner:
          $c_i \leftarrow \langle u_i, v_i, q_i \rangle = \langle u, v, Q(u, v) \rangle$ 
13:      Add  $c_i$  to Corners
14:  Sort Corners by  $q_i$  in descending order (strongest corners first)
15:  GoodCorners  $\leftarrow$  CLEANUPNEIGHBORS(Corners)
16:  return GoodCorners.

17: ISLOCALMAX(Q, u, v)    ▷ determine if  $Q(u, v)$  is a local maximum
18:   Let  $q_c \leftarrow Q(u, v)$  (center pixel)
19:   Let  $\mathcal{N} \leftarrow \text{Neighbors}(Q, u, v)$     ▷ values of all neighboring pixels
20:   if  $q_c \geq q_i$  for all  $q_i \in \mathcal{N}$  then
21:     return true
22:   else
23:     return false.

24: CLEANUPNEIGHBORS(Corners)  ▷ Corners is sorted by descending  $q$ 
25:   Create an empty list:
       GoodCorners  $\leftarrow []$ 
26:   while Corners is not empty do
27:      $c_i \leftarrow \text{REMOVEFIRST}(\text{Corners})$ 
28:     Add  $c_i$  to GoodCorners
29:     for all  $c_j$  in Corners do
30:       if  $\text{Dist}(c_i, c_j) < d_{\min}$  then
31:         Delete  $c_j$  from Corners
32:   return GoodCorners.

```

Prefilter (line 3): Smoothing with a small xy -separable filter

$$H_p = H_{px} * H_{py}, \text{ where}$$

$$H_{px} = \frac{1}{9} \begin{bmatrix} 2 & 5 & 2 \end{bmatrix} \quad \text{and} \quad H_{py} = H_{px}^T = \frac{1}{9} \begin{bmatrix} 2 \\ 5 \\ 2 \end{bmatrix}.$$

Gradient filter (line 4): Computing the first partial derivative in the x and y directions with

$$H_{dx} = \begin{bmatrix} -0.453014 & 0 & 0.453014 \end{bmatrix} \quad \text{and} \quad H_{dy} = H_{dx}^T = \begin{bmatrix} -0.453014 \\ 0 \\ 0.453014 \end{bmatrix}.$$

Blurfilter (line 6): Smoothing the individual components of the structure matrix M with separable Gaussian filters $H_b = H_{bx} * H_{by}$ with

$$H_{bx} = \frac{1}{64} \begin{bmatrix} 1 & 6 & 15 & 20 & 15 & 6 & 1 \end{bmatrix}, \quad H_{by} = H_{bx}^T = \frac{1}{64} \begin{bmatrix} 1 \\ 6 \\ 15 \\ 20 \\ 15 \\ 6 \\ 1 \end{bmatrix}.$$

Steering parameter (line 7): $\alpha = 0.04$ to 0.06 (default 0.05)

Response threshold (line 13): $t_H = 10,000$ to $1,000,000$ (default $25,000$)

Neighborhood radius (line 31): $d_{\min} = 10$ pixels

Morfolojik Filtreler (Süzgeçler)

Mathematical Morphology

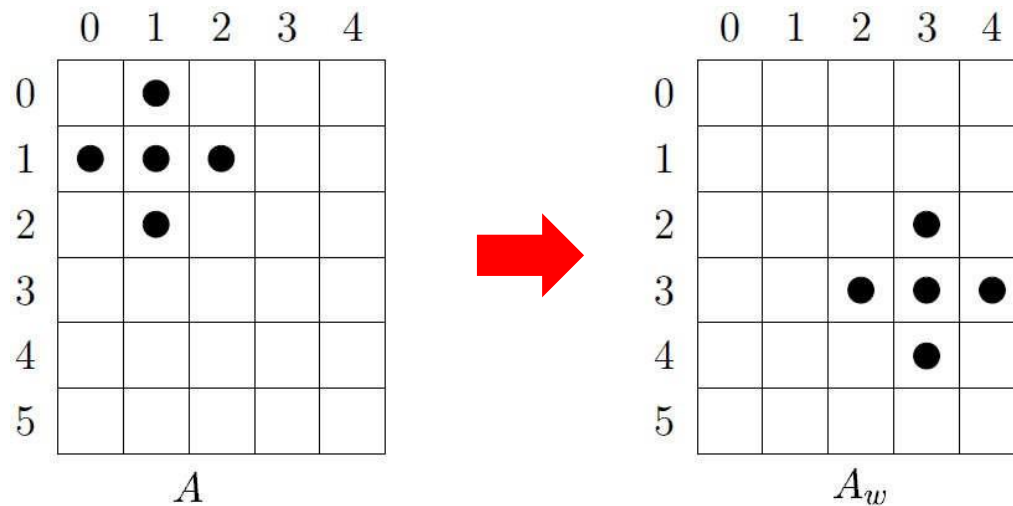
- Originally operated on Binary (black and white) images
- Binary images?
 - Faxes, digitally printed images
 - Obtained from thresholding grayscale images
- Morphological filters alter local structures in an image

Translation

- A is set of pixels in binary image
- $w = (x,y)$ is a particular coordinate point
- A is set A “translated” in direction (x,y) . i.e

$$A_x = \{(a,b) + (x,y) : (a,b) \in A\}.$$

- Example: If A is the cross-shaped set, and $w = (2,2)$

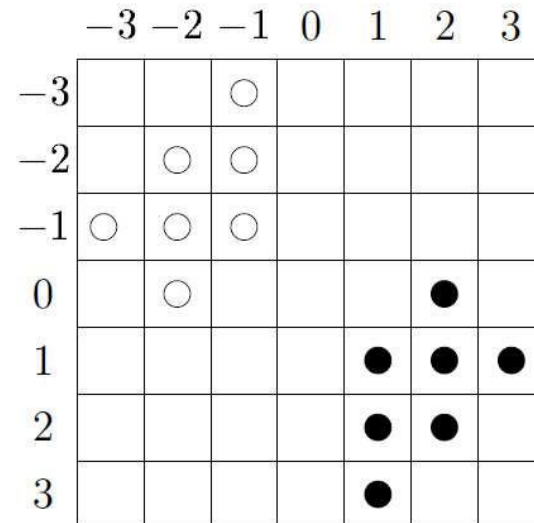


Reflection

- A is set of pixels
- Reflection of A is given by

$$\hat{A} = \{(-x, -y) : (x, y) \in A\}.$$

- An example of a reflection



Mathematical Morphology

- 2 basic mathematical morphology operations, (built from translations and reflections)
 - Dilation
 - Erosion

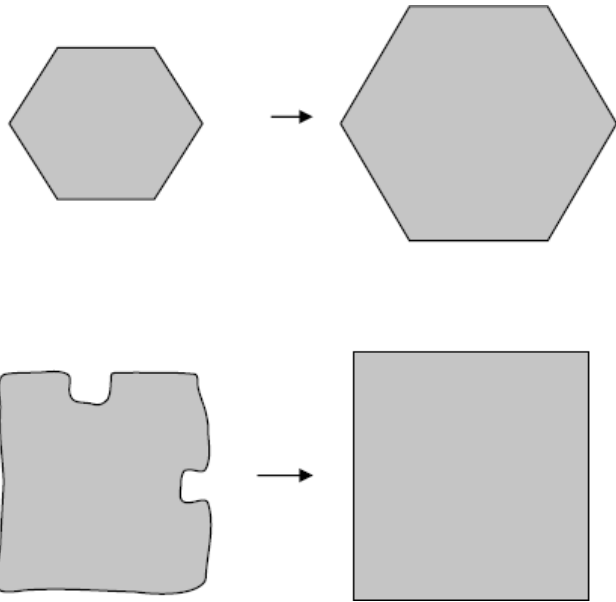
Also several composite relations

- Closing and Opening
- Conditional Dilation
- . . .

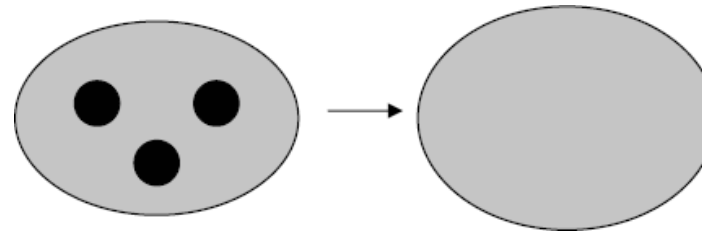
Dilation

- Dilation **expands** connected sets of 1s of a binary image. It can be used for

1. Growing features



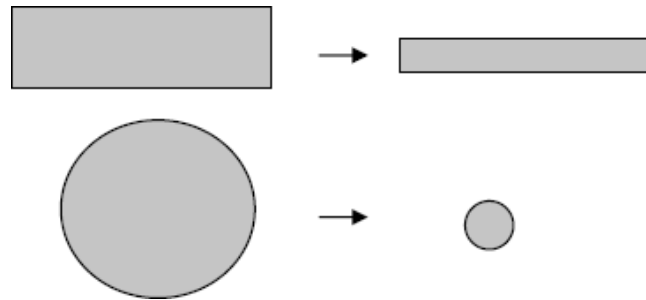
2. Filling holes and gaps



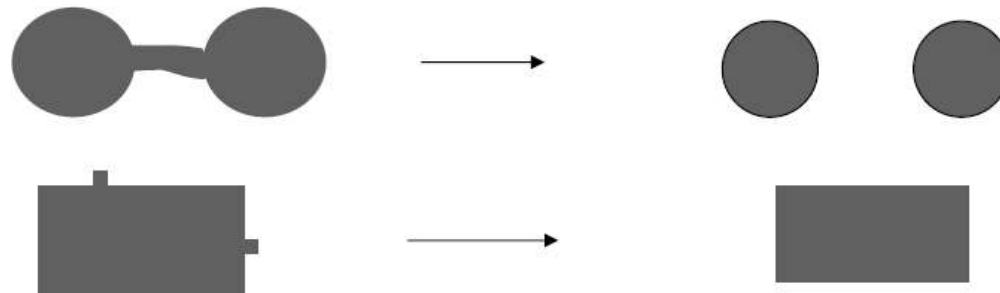
Erosion

- Erosion **shrinks** connected sets of 1s in binary image.
- Can be used for

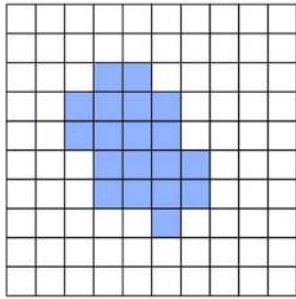
1. shrinking features



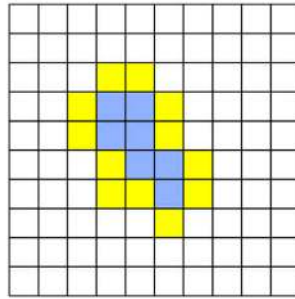
2. Removing bridges, branches and small protrusions



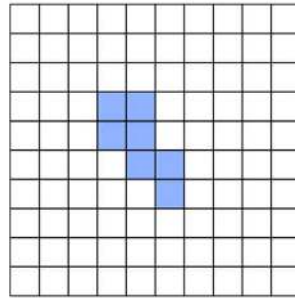
Shrink and Let Grow



(a)

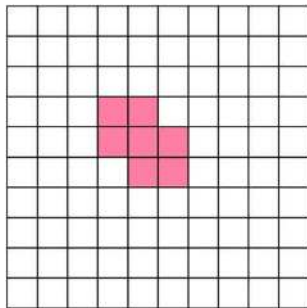


(b)

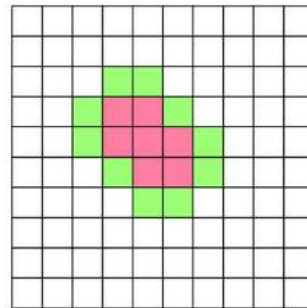


(c)

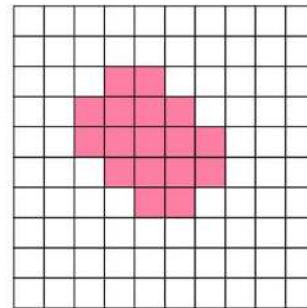
Shrinking: remove
border pixels



(a)



(b)

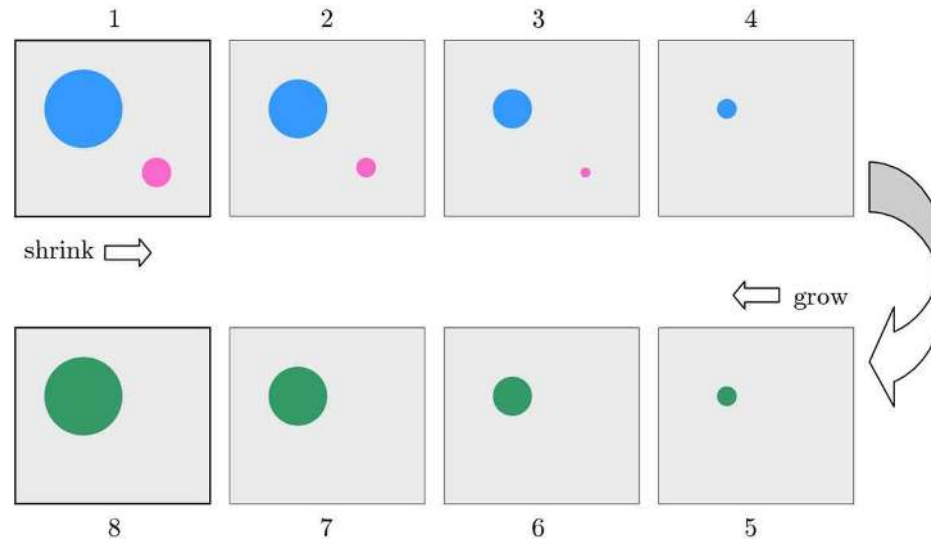


(c)

Growing: add layer
of pixels at border

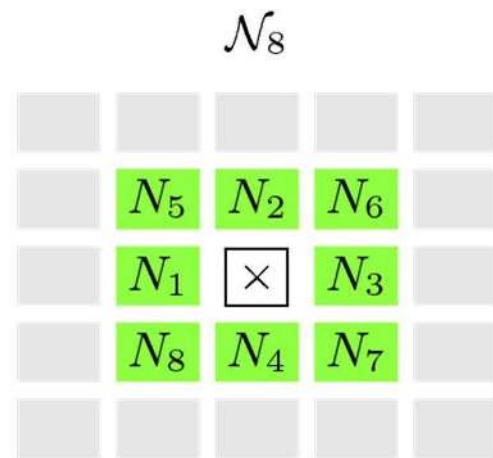
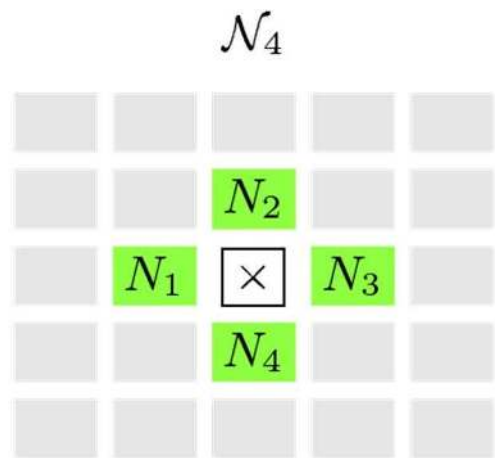
Shrinking and Let Grow

- Image structures are iteratively shrunk by peeling off a layer of thickness (layer of pixel) at boundaries
- Shrinking removes smaller structures, leaving only large structures
- Remaining structures are then grown back by same amount
- Eventually, large structures back to original size while smaller regions have disappeared



Basic Morphological Operations

- Definitions:
 - 4-Neighborhood (N_4): 4 pixels adjacent to given pixel in horizontal and vertical directions
 - 8-Neighborhood (N_8): : 4 pixels in N_4 + 4 pixels adjacent along diagonals





Formal Specification as Point Sets

- Morphological operations can be expressed by describing images as **2D point sets**
- For example, for a binary image ($I(u,v) \in \{0,1\}$)

$$\mathcal{Q}_I = \{p \mid I(p) = 1\}$$

- **Example:** OR operation union of individual sets

$$\mathcal{Q}_{I_1 \vee I_2} = \mathcal{Q}_{I_1} \cup \mathcal{Q}_{I_2}$$

Dilation

- Suppose A and B are sets of pixels, dilation of A by B

$$A \oplus B = \bigcup_{x \in B} A_x.$$

- Also called Minkowski addition. Meaning?
- Replace every pixel in A with copy of B (or vice versa)
- For every pixel x in B ,
 - Translate A by x
 - Take union of all these translations

$$A \oplus B = \{(x, y) + (u, v) : (x, y) \in A, (u, v) \in B\}.$$

Dilation Example

- For A and B shown below

$$B = \{(0,0), (1,1), (-1,1), (1,-1), (-1,-1)\}$$

	1	2	3	4	5
1					
2		●	●		
3		●	●		
4		●	●		
5		●	●	●	
6			●	●	
7					

A

	-1	0	1
-1	●		●
0		●	
1	●		●

B

	1	2	3	4	5
1					
2		■	■		
3		■	●	●	
4		■	●	●	
5		■	●	●	
6			●	●	●
7				●	●

$A_{(1,1)}$

Translation of A
by $(1,1)$

Dilation Example

	1	2	3	4	5
1			●	●	
2		■	●	●	
3		■	●	●	
4		■	●	●	●
5		■	■	●	●
6			■	■	
7					

$A_{(-1,1)}$

	1	2	3	4	5
1					
2		■	■		
3	●	●	■		
4	●	●	■		
5	●	●	■	■	
6	●	●	●	■	
7		●	●		

$A_{(1,-1)}$

	1	2	3	4	5
1	●	●			
2	●	●	■		
3	●	●	■		
4	●	●	●		
5		●	●	■	
6			■	■	
7					

$A_{(-1,-1)}$

	-1	0	1
-1	●		●
0		●	
1	●		●

B

	1	2	3	4	5
1	●	●	●	●	
2	●	■	●	●	
3	●	●	■	●	
4	●	●	●	■	●
5	●	●	●	●	■
6	●	●	■	■	●
7		●	●	●	●

$A \oplus B$

Union of all translations



Another Dilation Example

- Dilation increases size of structure
- A and B do not have to overlap
- **Example:** For the same A , if we change B to

$$B = \{(7, 3), (6, 2), (6, 4), (8, 2), (8, 4)\}$$

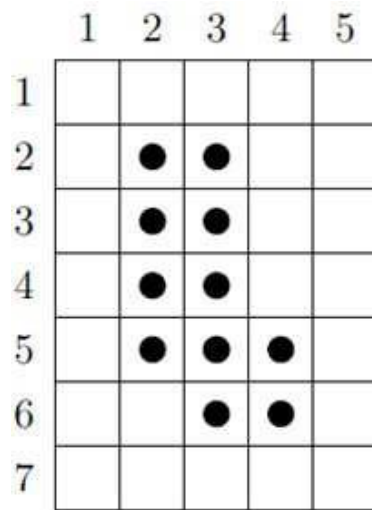
so that

$$A \oplus B = A_{(7,3)} \cup A_{(6,2)} \cup A_{(6,4)} \cup A_{(8,2)} \cup A_{(8,4)}$$

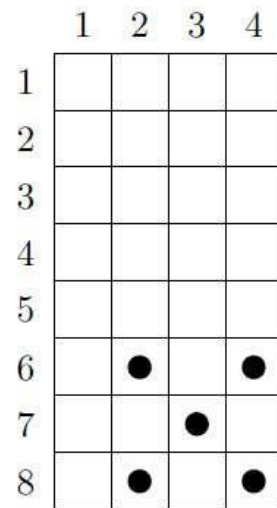
	1	2	3	4
1				
2				
3				
4				
5				
6		•		•
7			•	
8		•		•

B

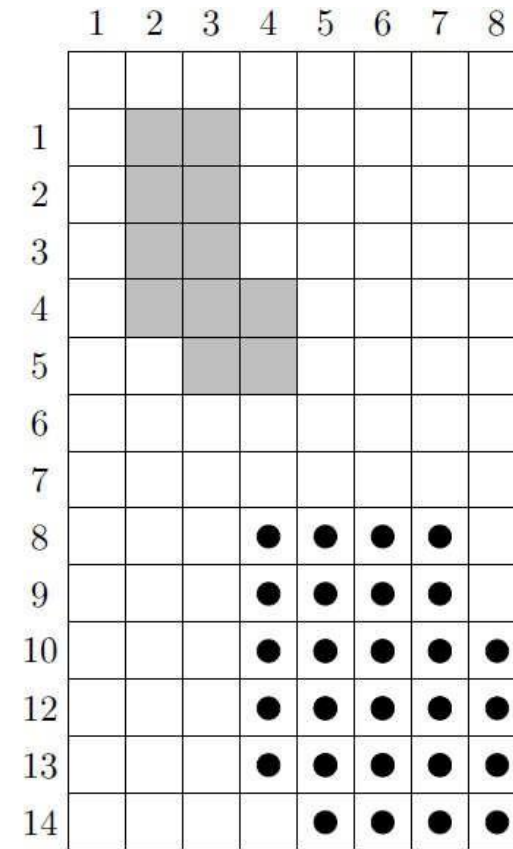
Another Dilation Example



A



B



$A \oplus B$

Example: Dilation of a Binary Image

Cross-Correlation Used
To Locate A Known
Target in an Image

Text Running
In Another
Direction

**Cross-Correlation Used
To Locate A Known
Target in an Image**

**Text Running
In Another
Direction**

Dilation

- We usually assume
 - A is being processed
 - B is a smaller set of pixels, called the **structuring element**

	1	2	3	4	5
1					
2		●	●		
3		●	●		
4		●	●		
5		●	●	●	
6			●	●	
7					

A

	1	2	3	4
1				
2				
3				
4				
5				
6		●		●
7			●	
8		●		●

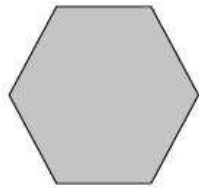
B

The Structuring Element

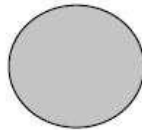
- A structuring element is a shape mask used in the basic morphological operations
- They can be any shape and size that is digitally representable, and each has an origin.



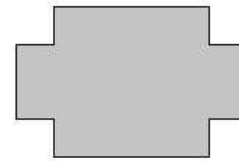
box



hexagon



disk

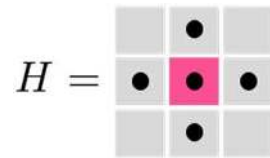


something

The Structuring Element

- Structuring element somewhat similar to a filter
- Contains only 0 and 1 values
- **Hot spot** marks origin of coordinate system of H
- **Example of structuring element:** 1-elements marked with •, 0-cells are empty

$$H(i, j) \in \{0, 1\}$$



 origin (hot spot)

Erosion

- Given sets A and B , the **erosion of A by B**

$$A \ominus B = \{w : B_w \subseteq A\}.$$

- Find all occurrences of B in A

	-1	0	1
-1		●	
0	●	●	●
1		●	

B

	1	2	3	4	5	6
1		●				
2	●	●	●	●	●	●
3			●	●	●	●
4			●	●		●
5		●	●	●	●	●
6			●	●		

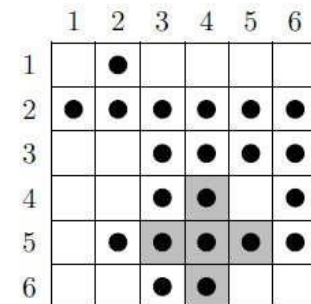
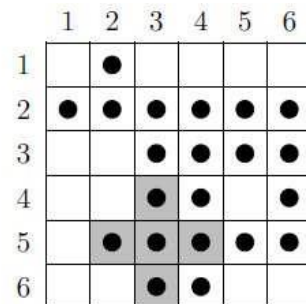
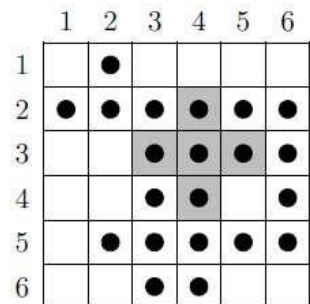
A

	1	2	3	4	5	6
1		●				
2	●	●	●	●	●	●
3			●	●	●	●
4			●	●		●
5		●	●	●	●	●
6			●	●		

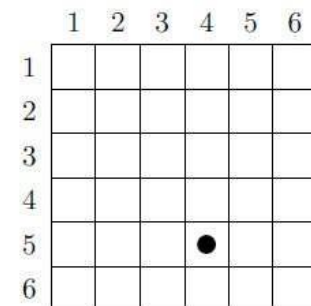
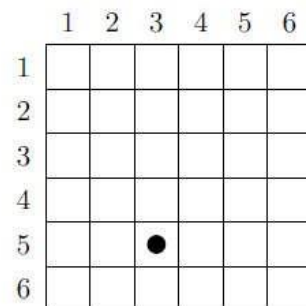
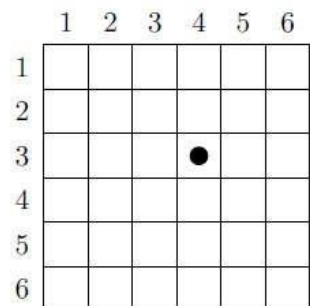
Example: 1 occurrence
of B in A

Erosion

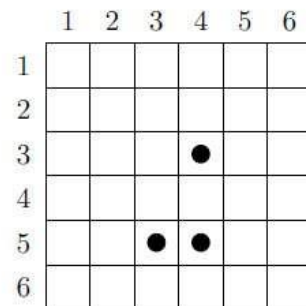
All occurrences
of B in A



For each
occurrences
Mark center of B



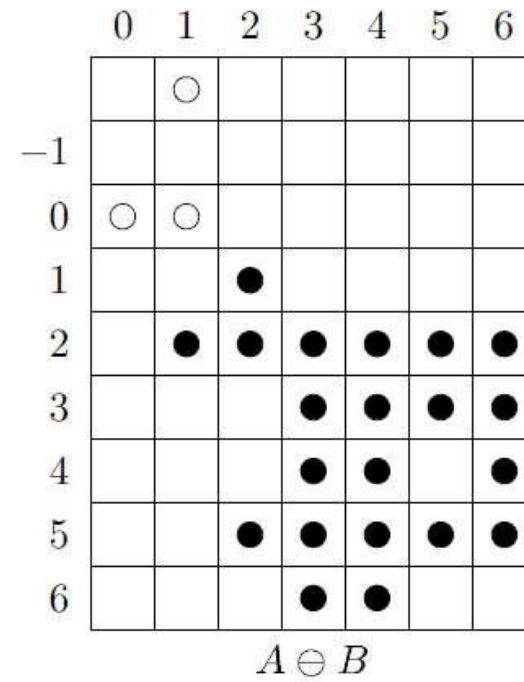
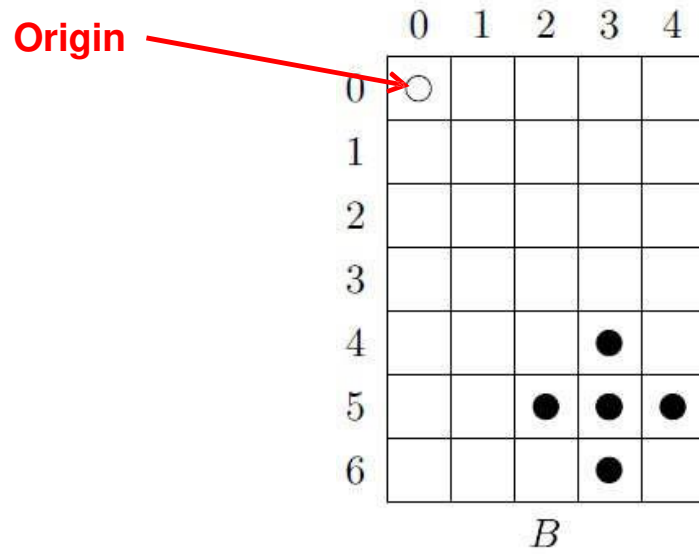
Erosion: union
of center of all
occurrences of
 B in A



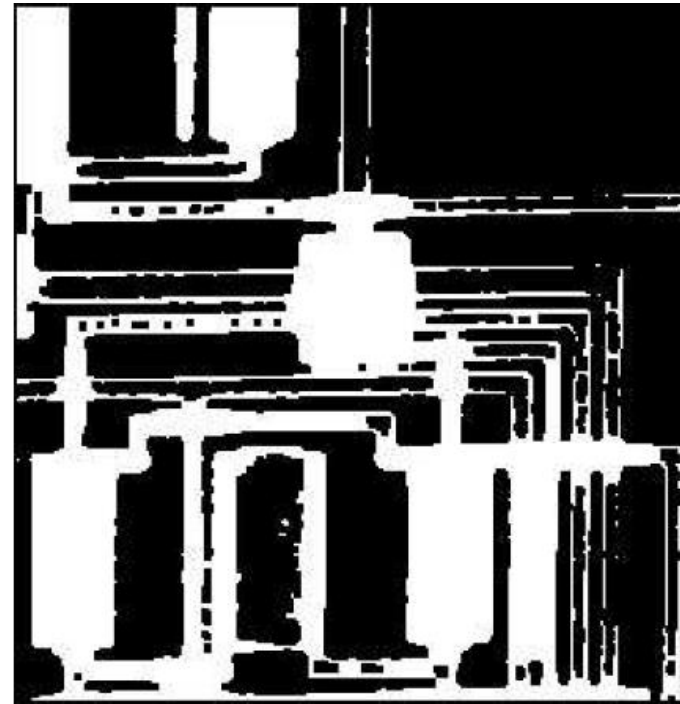
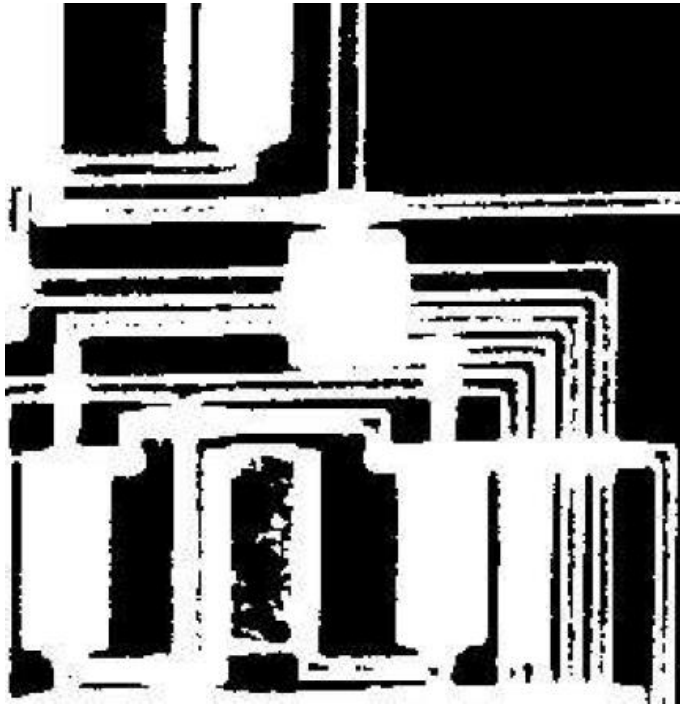
$$A \ominus B$$

Another Erosion Example

- The structuring element (B) does not have to contain the origin
- Another example where B does not contain the origin



Example: Erosion of Binary Image



Erosion

- Erosion related to **minkowski subtraction**

$$A - B = \bigcap_{b \in B} A_b.$$

- Erosion and dilation are **inverses** of each other
- It can be shown that

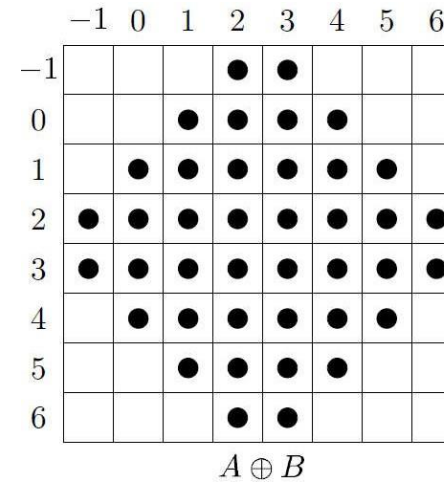
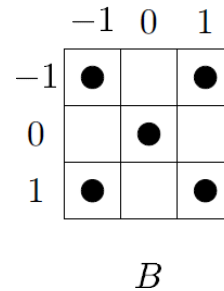
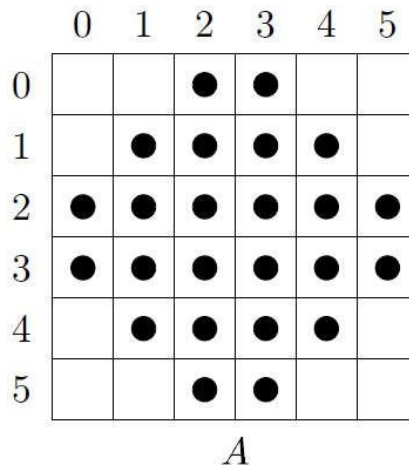
$$\overline{A \ominus B} = \overline{A} \oplus \hat{B}.$$

- And also that

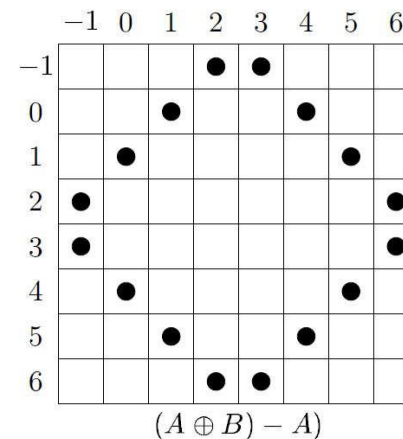
$$\overline{A \oplus B} = \overline{A} \ominus \hat{B}.$$

An Application: Boundary Detection

- Given an image A and structuring element B

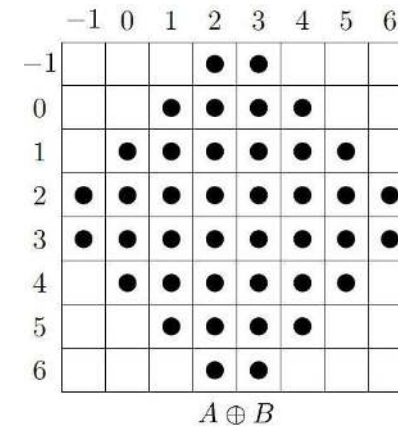
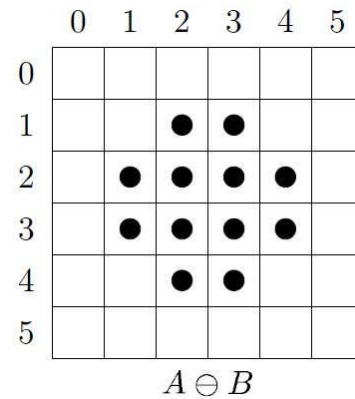


We can define **external boundary**



An Application: Boundary Detection

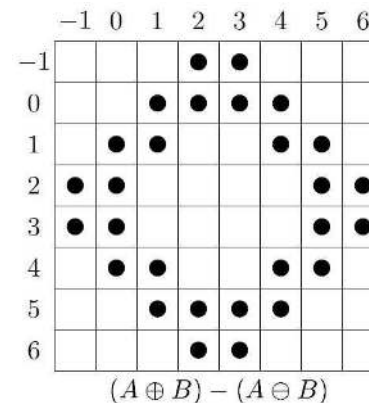
- Dilation of image A - erosion image A (by structuring element B)



- We can define morphological gradient

$$(A \oplus B) - (A \ominus B)$$

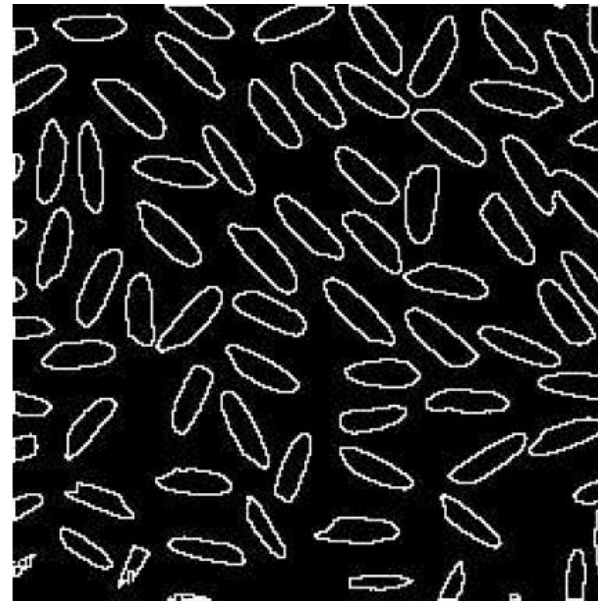
Morphological gradient = Dilation - erosion



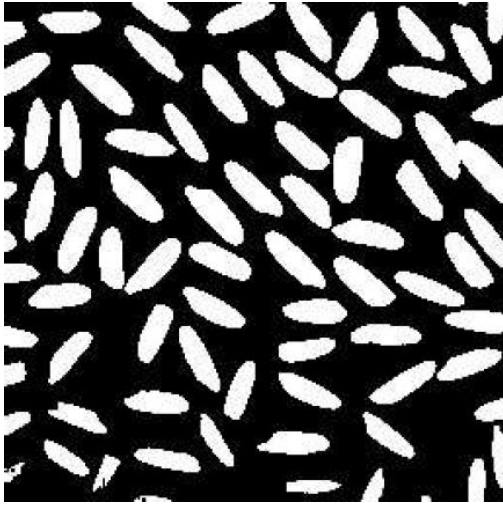
Example: Internal Boundary of Binary Image

- We can also define **internal boundary** as

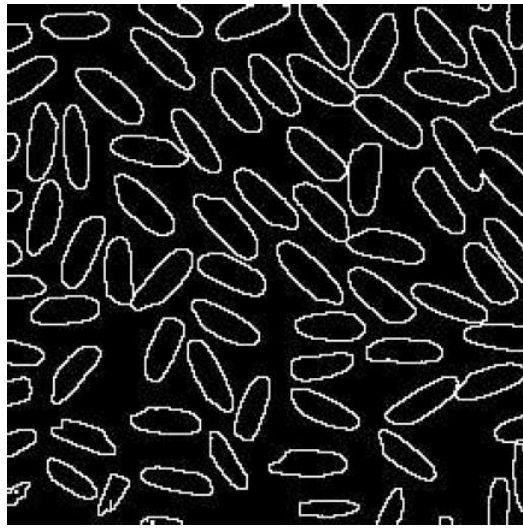
$$A - (A \ominus B)$$



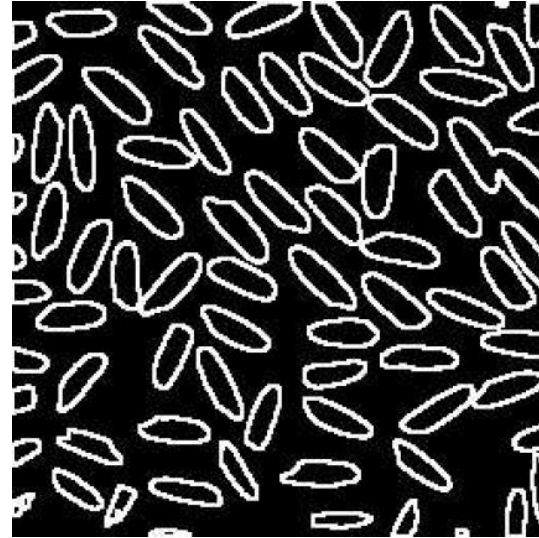
Example: External Boundary and Morphological Gradient



Image

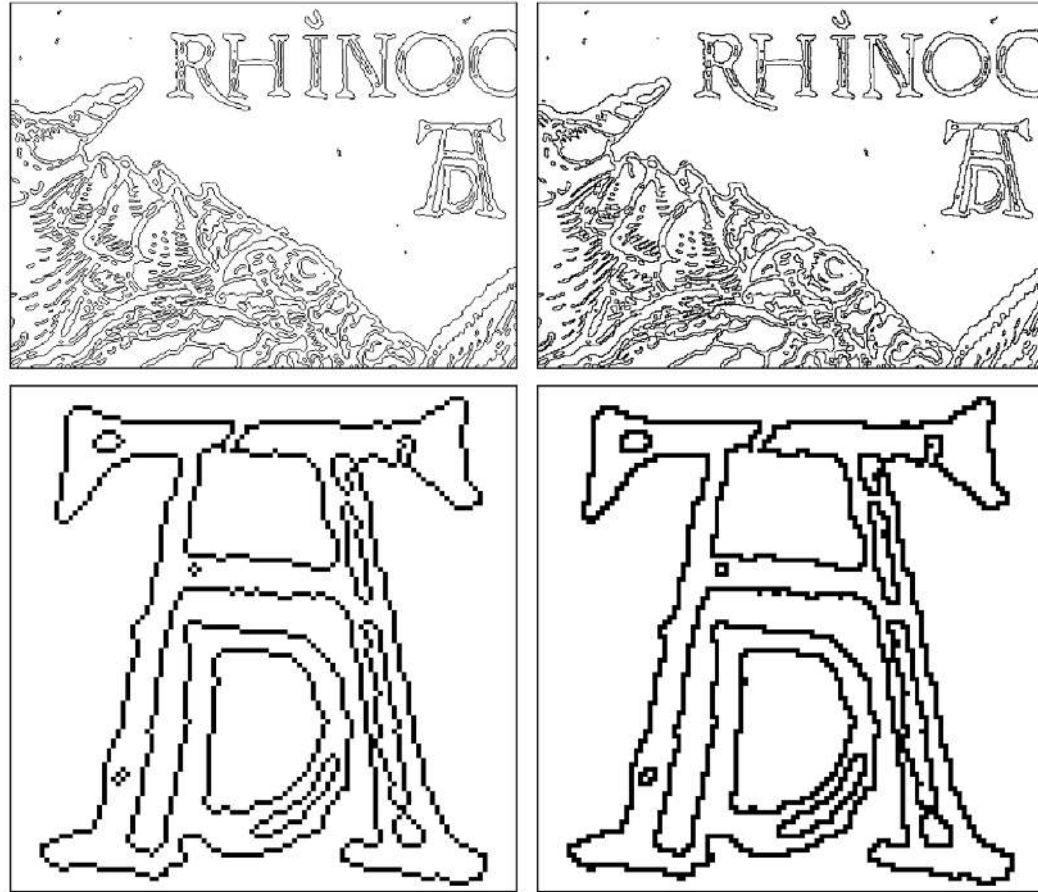


External Boundary



**Morphological
Gradient**

Example: Extraction of Boundary Pixels using Morphological Operations



(a)

(b)

Properties of Dilation

- Dilation operation is **commutative**

$$I \oplus H = H \oplus I$$

- Dilation is **associative** (ordering of applying it not important)

$$(I_1 \oplus I_2) \oplus I_3 = I_1 \oplus (I_2 \oplus I_3)$$

- Thus as with separable filters, more efficient to apply large structuring element as sequence of smaller structuring elements

$$I \oplus H_{\text{big}} = (\dots((I \oplus H_1) \oplus H_2) \oplus \dots \oplus H_K)$$

Properties of Erosion

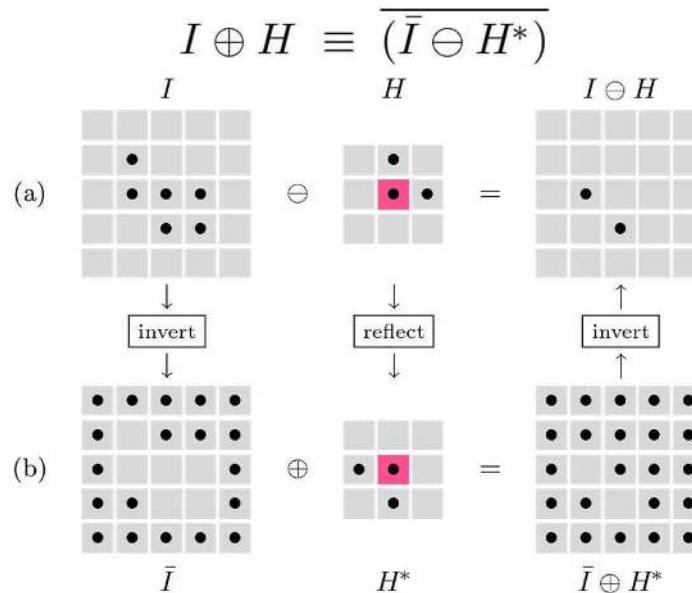
- Erosion is **not commutative**

$$I \ominus H \neq H \ominus I$$

- If erosion and dilation are combined, this chain rule holds

$$(I_1 \ominus I_2) \ominus I_3 = I_1 \ominus (I_2 \oplus I_3)$$

- Dilation of **foreground** = inverting (erosion of **background**)

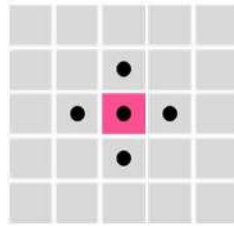


Dilation and Erosion Algorithm

```
1: DILATE ( $I, H$ )  
    $I$ : binary image of size  $w \times h$   
    $H$ : binary structuring element defined over region  $\mathcal{R}_H$   
   Returns the dilated image  $I' = I \oplus H$   
2:    $I' \leftarrow$  new binary image of size  $w \times h$   
3:    $I'(u, v) \leftarrow 0$ , for all  $(u, v)$   $\triangleright I' \leftarrow \emptyset$   
4:   for all  $(i, j) \in \mathcal{R}_H$  do  $\triangleright (i, j) = q$   
5:     if  $H(i, j) = 1$  then  $\triangleright q \in H$   
6:       MERGE THE SHIFTED  $I_q$  WITH  $I'$ :  $\triangleright I' \leftarrow I' \cup I_q$   
7:       for  $u \leftarrow 0 \dots (w-1)$  do  
8:         for  $v \leftarrow 0 \dots (h-1)$  do  $\triangleright (u, v) = p$   
9:           if  $I(u, v) = 1$  then  $\triangleright p \in I$   
10:             $I'(u+i, v+j) \leftarrow 1$   $\triangleright I' \leftarrow I' \cup (p+q)$   
11:   return  $I'$ .  
  
12: ERODE ( $I, H$ )  
13:    $\bar{I} \leftarrow \text{INVERT}(I)$   $\triangleright \bar{I} \leftarrow \neg I$   
14:    $H^* \leftarrow \text{REFLECT}(H)$   
15:   return  $\text{INVERT}(\text{DILATE}(\bar{I}, H^*))$ .  $\triangleright I \oplus H = \overline{(\bar{I} \oplus H^*)}$ 
```

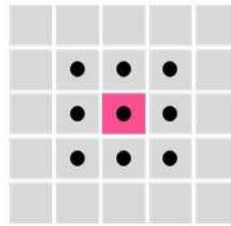
Designing Morphological Filters

- A morphological filter is specified by:
 - Type of operation (e.g. dilation, erosion)
 - Contents of structuring element



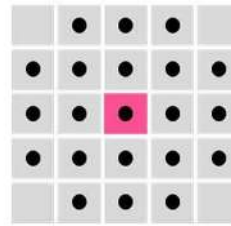
(a)

4-neighborhood



(b)

8-neighborhood



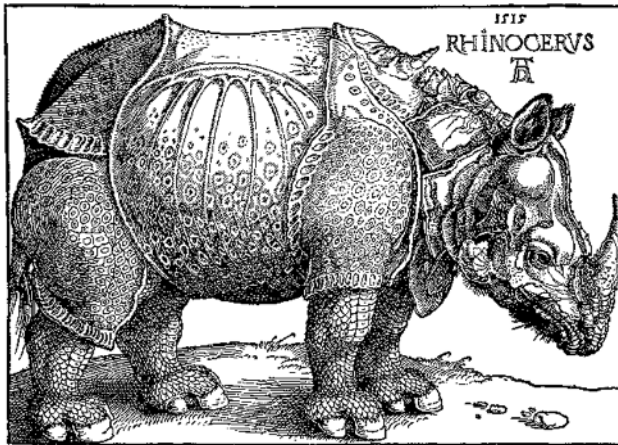
(c)

**Small Disk
(circular)**

- In practice, quasi-circular shaped structuring elements used
- Dilation with circular structuring of radius r adds thickness r
- Erosion with circular structuring of radius r removes thickness r

Example: Dilation and Erosion

- What if we erode and dilate the following image with disk-shaped structuring element?



Original image



Apply dilation and erosion
to this close up section

Dilation

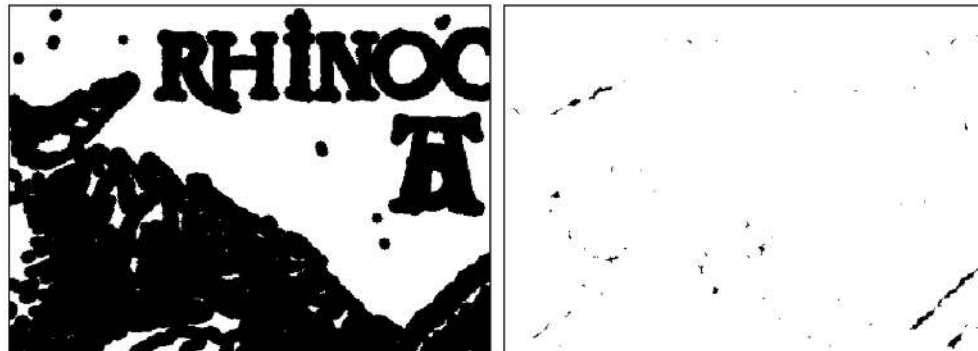
Erosion



$r = 1.0$



$r = 2.5$



$r = 5.0$

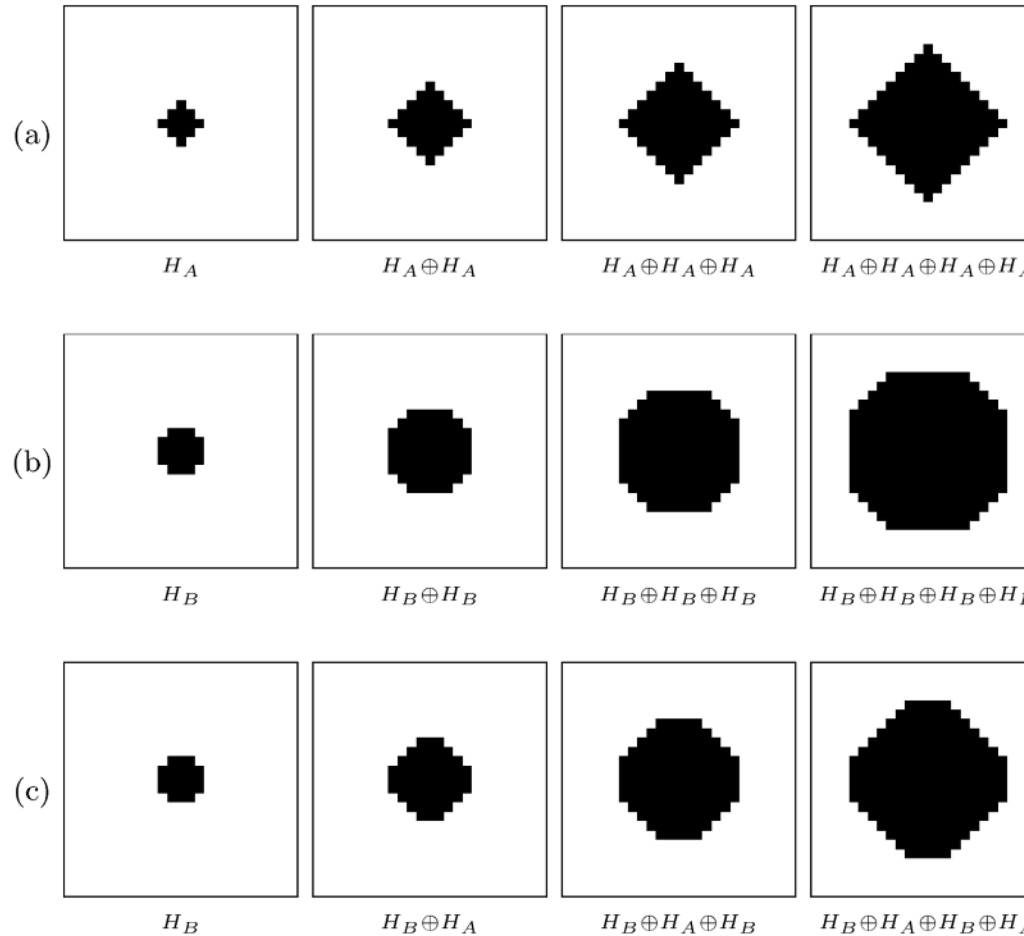
Example: Dilation and Erosion



Dilation and Erosion
using Different
Structuring
Elements

Example: Composing Large Filters by Repeatedly Applying Smaller Filters

- More efficient
- E.g. composing Isotropic filter



Opening

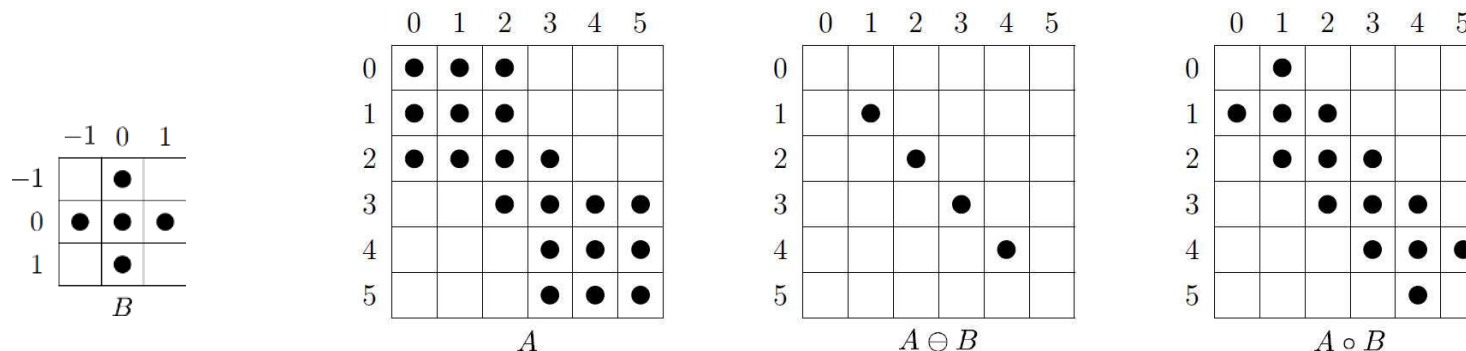
- Opening and closing: operations built on dilation and erosion
- Opening of A by structuring element B

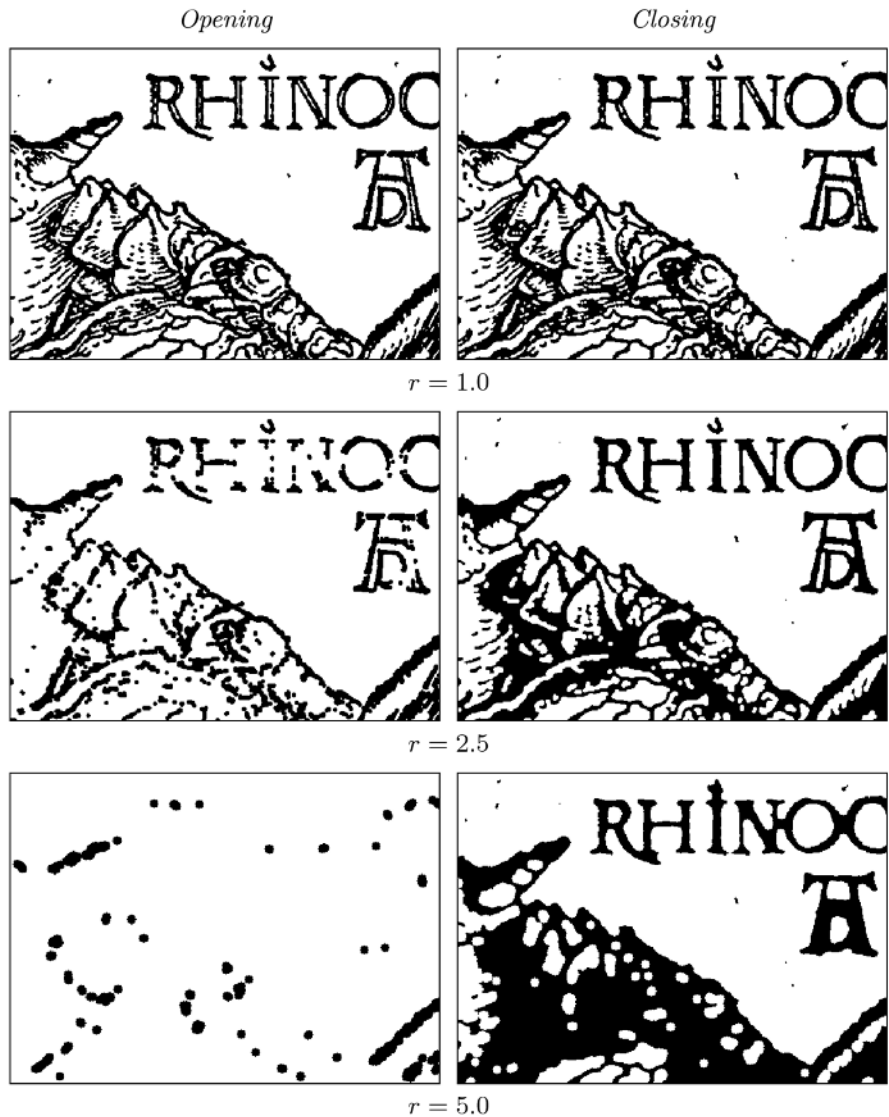
$$A \circ B = (A \ominus B) \oplus B.$$

- i.e. opening = erosion followed by dilation. Alternatively

$$A \circ B = \cup \{B_w : B_w \subseteq A\}.$$

- i.e. Opening = union of all translations of B that fit in A
- Note: Opening includes all of B , erosion includes just (0,0) of B



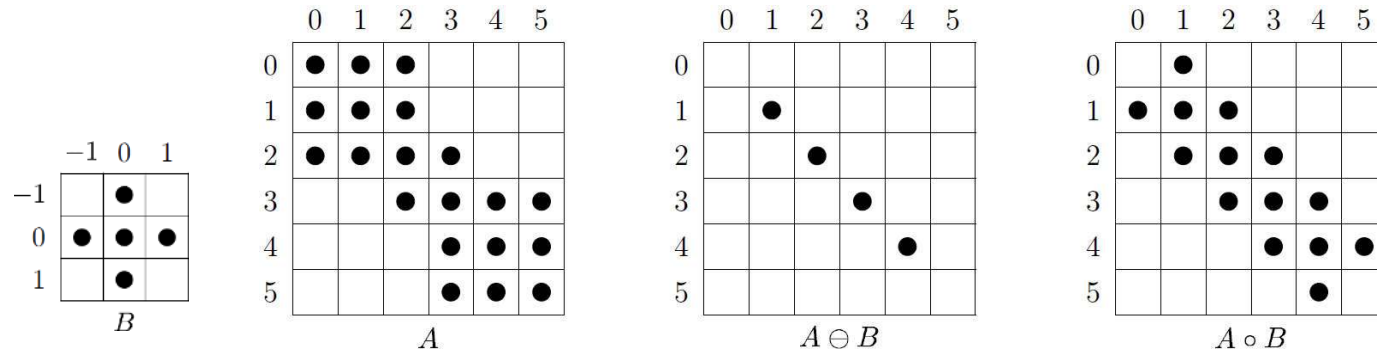


**Binary opening and closing with disk-shaped
Structuring elements of radius $r = 1.0, 2.5, 5.0$**

Opening

- g. All foreground structures smaller than structuring element are eliminated by first step (erosion)
 - Remaining structures smoothed by next step (dilation) then grown back to their original size

Properties of Opening



1. $(A \circ B) \subseteq A$: Opening is subset of A (not the case with erosion)
2. $(A \circ B) \circ B = A \circ B$: Can apply opening only once, also called **idempotence** (not the case with erosion)
3. Subsets: If $A \subseteq C$, then $(A \circ B) \subseteq (C \circ B)$.
4. Opening tends to smooth an image, break narrow joins, and remove thin protrusions.

Closing

- Closing of A by structuring element B

$$A \bullet B = (A \oplus B) \ominus B.$$

- i.e. closing = dilation followed by erosion

	-1	0	1
-1		●	
0	●	●	●
1		●	

B

	0	1	2	3	4	5	6
0							
1		●	●	●		●	
2		●		●			
3		●	●	●		●	
4							
5		●	●	●	●		
6							

A

	0	1	2	3	4	5	6
0		●	●	●		●	
1	●	●	●	●	●	●	●
2	●	●	●	●	●	●	
3	●	●	●	●	●	●	●
4		●	●	●	●	●	
5	●	●	●	●	●	●	
6		●	●	●	●	●	

$A \oplus B$

	0	1	2	3	4	5	6
0							
1		●	●	●		●	
2		●	●	●	●		
3		●	●	●	●	●	
4			●	●	●		
5		●	●	●	●		
6							

$A \bullet B$

Properties of Closing

1. Subset: $A \subseteq (A \bullet B)$.
2. **Idempotence:** $(A \bullet B) \bullet B = A \bullet B$;
3. Also If $A \subseteq C$, then $(A \bullet B) \subseteq (C \bullet B)$.
4. Closing tends to:
 - a) Smooth an image
 - b) Fuse narrow breaks and thin gulfs
 - c) Eliminates small holes.

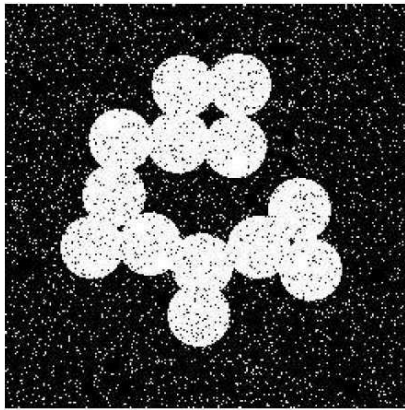
An Example of Closing



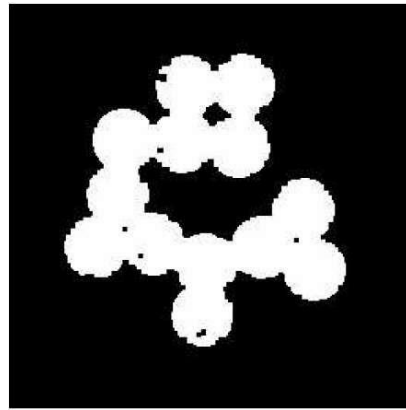
Cross-Correlation Used
To Locate A Known
Target in an Image

Text Running
In Another
Direction

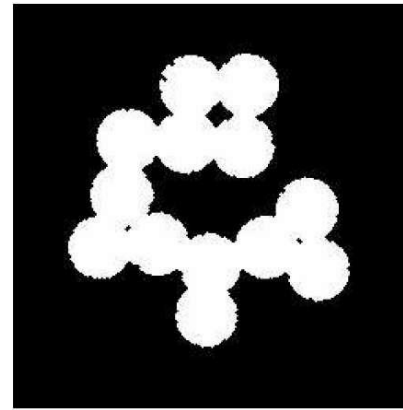
Noise Removal: Morphological Filtering



(a)



(b)



(c)

(b) Filter once
(c) Filter Twice

- First dilation returns the holes to their original size
- Second dilation removes the holes but enlarges objects in image
- To reduce them to their correct size, perform a final erosion:

$$(((A \ominus B) \oplus B) \oplus B) \ominus B.$$

- Inner 2 operations = opening, Outer 2 operations = closing.
- This noise removal method = opening followed by closing

$$(A \circ B) \bullet B).$$

Relationship Between Opening and Closing

- Opening and closing are duals
 - i.e. Opening foreground = closing background, and vice versa
- Complement of an opening = the closing of a complement

$$\overline{A \bullet B} = \overline{A} \circ \hat{B}$$

- Complement of a closing = the opening of a complement.

$$\overline{A \circ B} = \overline{A} \bullet \hat{B}.$$

Grayscale Morphology

- Morphology operations can also be applied to grayscale images
- Just replace (OR, AND) with (MAX, MIN)
- Consequently, morphology operations defined for grayscale images can also operate on binary images (but not the other way around)
 - ImageJ has single implementation of morphological operations that works on binary and grayscale
- For color images, perform grayscale morphology operations on each color channel (RGB)
- For grayscale images, structuring element contains real values
- Values may be –ve or 0

Grayscale Morphology

- Elements in structuring element that have value 0 do contribute to result
- Design of structuring elements for grayscale morphology must distinguish between 0 and empty (don't care)

0	1	0
1	2	1
0	1	0

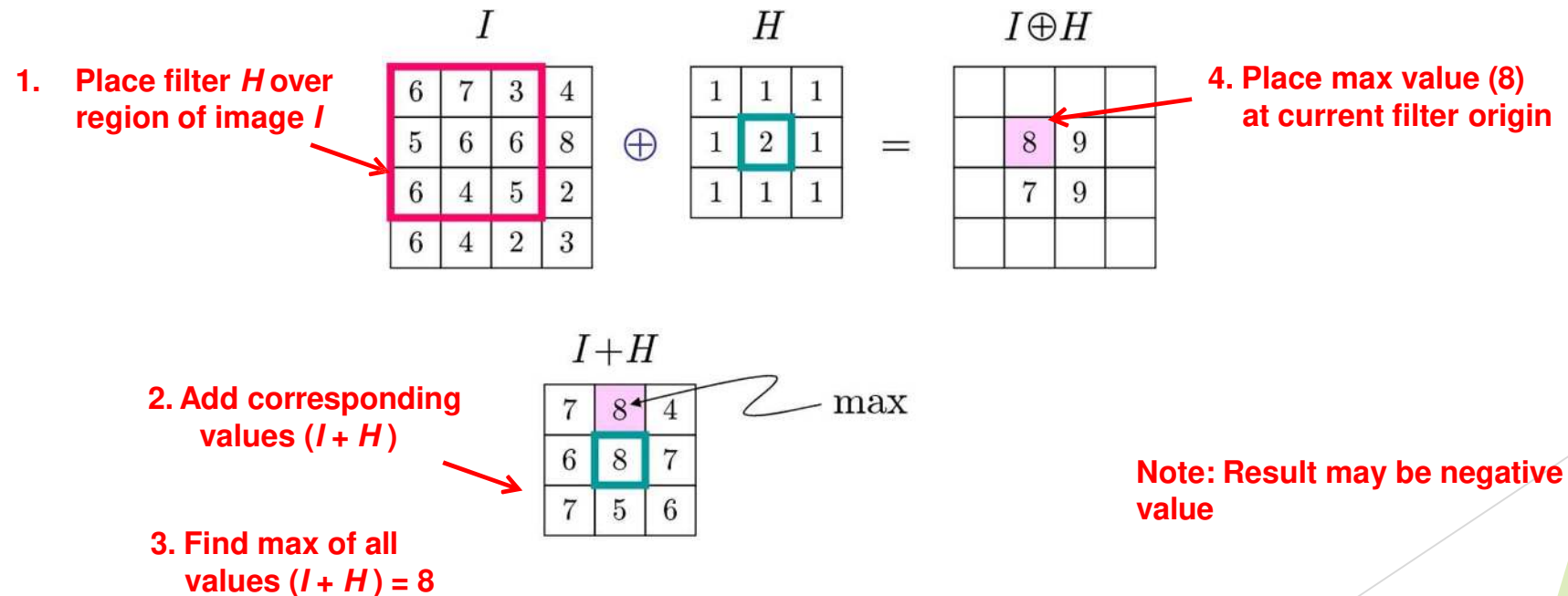
 $\neq$

	1	
1	2	1
	1	

Grayscale Dilation

- **Grayscale dilation:** Max (value in filter H + image region)

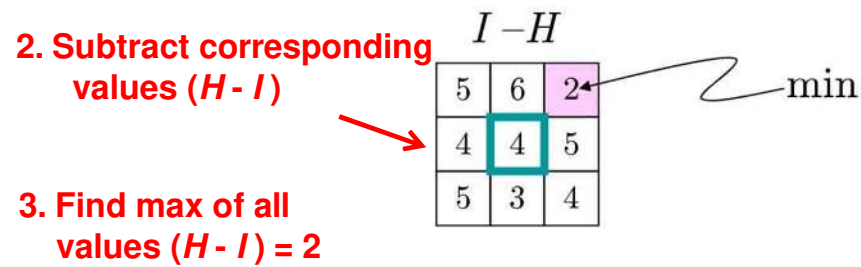
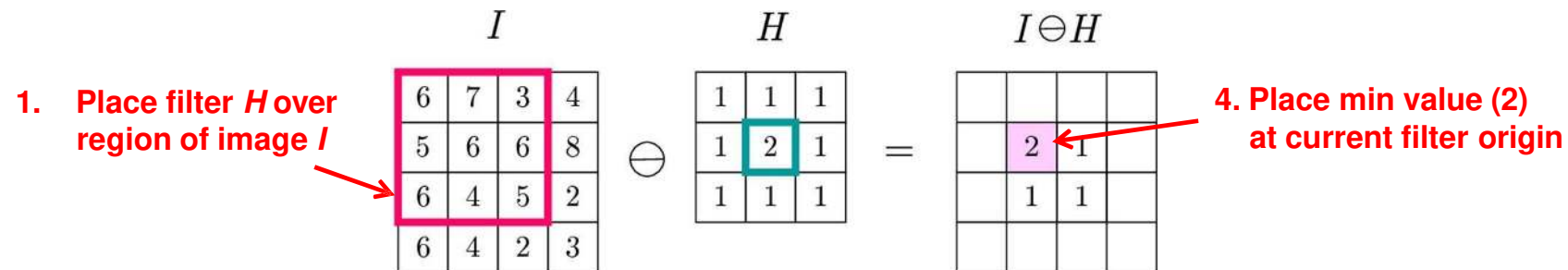
$$(I \oplus H)(u, v) = \max_{(i,j) \in H} \{I(u+i, v+j) + H(i, j)\}$$



Grayscale Erosion

- **Grayscale erosion:** Min (value in filter H + image region)

$$(I \ominus H)(u, v) = \min_{(i,j) \in H} \{I(u+i, v+j) - H(i, j)\}$$



Note: Result may be negative value

Grayscale Opening and Closing

- **Recall:** Opening = erosion then dilation:
- So we can implement grayscale opening as:
 - Grayscale erosion then grayscale dilation

- **Recall:** Closing = dilation then erosion:
- So we can implement grayscale erosion as:
 - Grayscale dilation then grayscale erosion

Grayscale Dilation and Erosion

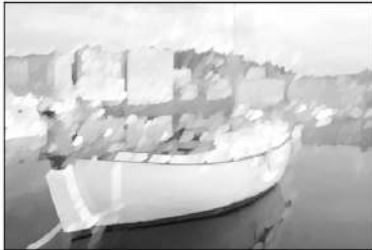


- Grayscale dilation and erosion with disk-shaped structuring elements of radius $r = 2.5, 5.0, 10.0$

H

Dilation

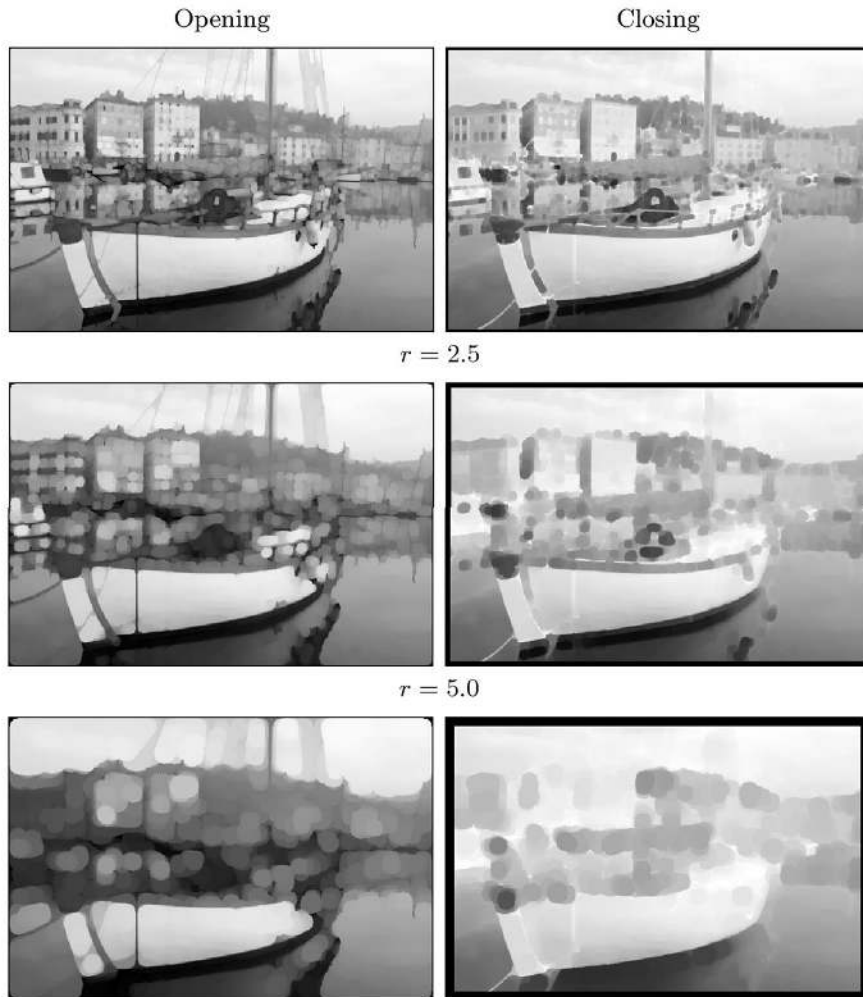
Erosion



Grayscale Dilation and Erosion

- Grayscale dilation and erosion with various free-form structuring elements

Grayscale Opening and Closing



- Grayscale opening and closing with disk-shaped structuring elements of radius $r = 2.5, 5.0, 10.0$

Implementing Morphological Filters

- Morphological operations implemented in ImageJ as methods of class `ImageProcessor`
 - `dilate( )`
 - `erode( )`
 - `open( )`
 - `close( )`
- The class `BinaryProcessor` offers these morphological methods
 - `outline( )`
 - `skeletonize( )`

Implementation of ImageJ dilate()

```
3  ...
4  void dilate(ImageProcessor I, int[] [] H){
5      //assume that the hot spot of H is at its center (ic,jc):
6      int ic = (H[0].length-1)/2;
7      int jc = (H.length-1)/2;
8
9      //create a temporary (empty) image:
10     ImageProcessor tmp
11         = I.createProcessor(I.getWidth(),I.getHeight());
12
13     for (int j=0; j<H.length; j++){
14         for (int i=0; i<H[j].length; i++){
15             if (H[j][i] > 0) { // this pixel is set
16                 //copy image into position (i-ic,j-jc):
17                 tmp.copyBits(I,i-ic,j-jc,Blitter.MAX);
18             }
19         }
20     }
21     //copy the temporary result back to original image
22     I.copyBits(tmp,0,0,Blitter.COPY);
23 }
```

Center of filter H
assumed to be at center

Create temporary copy
of image

Perform dilation by
copying shifted version
of original into tmp

Replace original image
destructively with tmp image

Implementation of ImageJ Erosion

- Erosion implementation can be derived from dilation
- **Recall:** Erosion is dilation of background
- So invert image, perform dilation, invert again

```
24 void erode(ImageProcessor I, int[][] H) {  
25     ip.invert();  
26     dilate(ip, reflect(H));  
27     ip.invert();  
28 }
```

Implementation of Opening and Closing

- **Recall:** Opening = erosion then dilation:

```
29 void open(ImageProcessor I, int[] [] H) {  
30     erode(I,H);  
31     dilate(I,H);  
32 }
```

- **Recall:** Closing = dilation then erosion:

```
33 void close(ImageProcessor I, int[] [] H) {  
34     dilate(I,H);  
35     erode(I,H);  
36 }
```